

PROFIND TRAINING · TEST ET QUALITÉ DES APPLICATIONS

Tests automatisés Web avec Playwright et intelligence artificielle

Automatiser, fiabiliser et industrialiser des tests end-to-end sur des applications web modernes.

DURÉE

2 jours, 14 heures

DATES

24 et 25 septembre 2026

LIEU

Paris La Défense

NIVEAU

Débutant

FORMATEUR

Mehdi Tajmouati

Tour de table

Votre contexte

- Votre rôle : développement, QA, ops, produit ?
- Les applications que vous testez aujourd'hui
- Vos outils actuels : Selenium, Cypress, tests manuels, rien ?

Vos attentes

- Ce qui vous coûte le plus cher aujourd'hui en test
- Votre pratique de JavaScript / TypeScript
- Votre usage actuel de l'IA au quotidien

Les contenus sont ajustés en cours de route selon les besoins identifiés pendant la formation.

Objectifs de la formation

1 • Automatiser

Écrire des tests end-to-end avec Playwright sur des applications web modernes.

2 • Fiabiliser

Concevoir des scénarios fiables grâce à des sélecteurs robustes et des assertions adaptées.

3 • Maintenir

Structurer, maintenir et déboguer une suite de tests dans un contexte professionnel.

4 • Accélérer avec l'IA

Exploiter l'intelligence artificielle pour générer, corriger et optimiser des tests.

Les quatre modules

#	MODULE	CE QUE VOUS EN RETIREZ
1	Comprendre les tests automatisés et découvrir Playwright	Un environnement qui tourne et un premier parcours automatisé.
2	Écrire des tests web fiables et maintenables	Des tests qui ne cassent pas au premier changement de CSS.
3	Structurer, maintenir et déboguer une suite	Savoir pourquoi un test échoue en moins de deux minutes.
4	Industrialiser et intégrer dans le cycle de développement	Une suite qui tourne toute seule à chaque commit.

L'IA n'est pas un module à part : elle intervient dans les modules 2, 3 et 4, à chaque fois sur une tâche précise.

Le fil des deux journées

Jour 1 : écrire des tests qui marchent, et qui continuent de marcher

MATIN

Pourquoi automatiser, comment Playwright fonctionne, premier test

TP 1

Un parcours utilisateur automatisé de bout en bout

APRÈS-MIDI

Sélecteurs robustes, assertions, Codegen, génération par IA

TP 2

Trois tests, dont un généré et corrigé

Jour 2 : passer d'un test à une suite exploitée par l'équipe

MATIN

Organisation, isolation, Page Objects, outils de débogage

TP 3

Une suite instable rendue fiable

APRÈS-MIDI

Mocking réseau, fixtures, multi-navigateurs, CI/CD

TP 4

Une suite complète branchée en intégration continue

Chaque demi-journée suit le même rythme : un apport court, puis un TP qui produit quelque chose d'utilisable.

Déroulé des deux journées

Jour 1 : jeudi 24/09

09h00	Accueil, tour de table, objectifs
09h30	Module 1 : Tests automatisés et Playwright
11h00	TP 1 : Premier parcours utilisateur
12h30	Déjeuner
14h00	Module 2 : Tests fiables et maintenables
16h00	TP 2 : Tests assistés par IA
17h30	Fin de journée

Jour 2 : vendredi 25/09

09h00	Retour sur le jour 1, questions
09h30	Module 3 : Structurer et déboguer
11h00	TP 3 : Fiabiliser une suite existante
12h30	Déjeuner
14h00	Module 4 : Industrialiser, mocking, CI/CD
16h00	TP 4 : Suite complète et intégration continue
17h15	Bilan, ressources, évaluation
17h30	Fin de la formation

Modalités pédagogiques

Alternance

Apport théorique court, puis mise en pratique immédiate. Environ la moitié du temps sur le clavier.

Cas concrets

Méthodes expositive, active et participative. Réflexion et échanges sur cas pratiques, retours d'expérience.

Évaluation continue

Le formateur évalue la progression au fil des TP, études de cas et mises en situation.

Côté administratif

- Feuille de présence à signer **par demi-journée**, soit quatre signatures sur les deux jours.
- Attestation de fin de formation remise à chaque participant.
- Support de cours numérique transmis en fin de formation.

Comment lire ces supports

Les encadrés

Un point à retenir, une règle, une clarification utile.

Un piège fréquent, une erreur qui coûte cher.

Bonne pratique

Ce qu'il faut faire, et pourquoi.

Les repères de niveau

Base Le socle du programme, vu par tout le monde.

Avancé Un approfondissement, traité selon le rythme du groupe. Vous le retrouverez dans le bloc d'annexes, à la fin du module.

Si le temps le permet Une slide qui approfondit un sujet déjà vu. Si nous la sautons, vous ne perdez rien du programme : elle est dans le support.

Le code

Tous les exemples sont en **TypeScript**. Ils sont exécutables tels quels, sauf mention contraire.

Le support numérique complet vous est transmis en fin de formation : inutile de recopier le code.

Vérifions les environnements

Trois commandes, maintenant

```
node --version      # attendu : v18 ou plus
npm --version
git --version
```

Puis, dans un dossier de travail vide

```
npm init playwright@latest
```

Le téléchargement des navigateurs peut être bloqué par le proxy d'entreprise : signalez-le tout de suite, une archive locale est prévue.

Points de blocage fréquents

- Proxy sortant qui bloque `npx playwright install`
- Droits insuffisants pour installer Node.js
- Antivirus qui met les navigateurs en quarantaine
- Accès à l'assistant IA non autorisé sur le poste

Chaque blocage a une solution de repli. On règle tout maintenant plutôt qu'au milieu du premier TP.

Le JavaScript dont vous aurez besoin

Playwright n'exige pas d'être développeur JavaScript. Quatre notations suffisent pour lire et écrire tous les tests des deux jours.

1. La fonction fléchée

```
// une fonction anonyme, notation courte
async ({ page }) => {
  // corps du test
}
```

2. La déstructuration

```
// on extrait la propriété « page » de l'objet reçu
async ({ page, browserName }) => { ... }

const { test, expect } = require('@playwright/test');
```

3. Le littéral de gabarit

```
const email = `utilisateur-${Date.now()}@test.fr`;

await page.goto(`/produits/${id}`);
```

4. L'objet en argument nommé

```
page.getByRole('button', { name: 'Valider' });

await expect(loc).toBeVisible({ timeout: 10000 });
```

Si l'une de ces quatre notations vous est étrangère, dites-le maintenant : elles reviennent dans chaque exemple.

async et await, en une slide

Toute action de Playwright prend du temps : cliquer suppose d'attendre que le navigateur réponde. Ces actions renvoient donc une **promesse**, et `await` signifie « attends le résultat avant de continuer ».

Sans await

```
page.getByLabel('Email').fill('a@b.fr');  
page.getByRole('button').click();  
// les deux partent en même temps,  
// le test se termine avant elles
```

Avec await

```
await page.getByLabel('Email').fill('a@b.fr');  
await page.getByRole('button').click();  
// chaque ligne attend la précédente
```

Une assertion sur une valeur **déjà calculée** reste synchrone : `expect(total).toBe(3)` s'écrit sans `await`.

La règle

`await` devant chaque **action** Playwright, et devant les assertions portant sur une page, un locator ou une réponse.

Comment repérer un oubli

- Le test passe alors qu'il ne devrait pas
- Le test est anormalement rapide
- Un avertissement *promise was not awaited*

Le linter ESLint de Playwright les signale automatiquement.

MODULE 1

Comprendre les tests automatisés et découvrir Playwright

Pourquoi automatiser, ce que Playwright apporte, et un premier test qui tourne.

MODULE 1 • JOUR 1 MATIN

Comprendre les tests automatisés et découvrir Playwright

DURÉE

≈ 1 h 30 + TP

À LA FIN

Un environnement opérationnel et un parcours automatisé

Ce que couvre ce module

- Les principes des tests logiciels et des tests **end-to-end**
- Les différences entre tests **manuels** et tests **automatisés**
- Les fonctionnalités et les usages de **Playwright** pour les applications web
- Installer et configurer un environnement de tests automatisés
- Automatiser les premières actions navigateur : **navigation, clics et saisie**

Travaux pratiques : automatisation d'un parcours utilisateur simple sur une application web.

Le vocabulaire du test automatisé

TERME	CE QU'IL DÉSIGNE
Suite	L'ensemble des tests d'un projet.
Spec	Un fichier de tests. Par convention <code>panier.spec.ts</code> .
Cas de test	Un scénario unique, déclaré par un appel à <code>test()</code> .
Assertion	La vérification qui décide si le test passe ou échoue.
Fixture	Une ressource préparée avant le test et nettoyée après : un onglet, une session connectée, un jeu de données.
Flaky	Un test qui échoue puis réussit sans qu'on ait rien changé.
Régression	Une fonctionnalité qui marchait et qui ne marche plus.
Smoke test	Le petit lot de tests qui vérifie que l'essentiel tient debout.
Headless	Le navigateur tourne sans fenêtre visible. Le mode par défaut.

Pourquoi tester ?

Un défaut coûte d'autant plus cher qu'il est découvert tard. Le même bug corrigé à l'écriture du code, à la recette, ou en production, ne mobilise ni les mêmes personnes ni les mêmes délais.

DÉCOUVERT...	CE QU'IL FAUT MOBILISER
À l'écriture	Le développeur, quelques minutes
En revue	Deux personnes, un aller-retour
En recette	QA, développeur, un cycle de correction
En production	Support, correctif urgent, communication, parfois un impact client

Ce qu'apporte un test automatisé

- Il rejoue le même scénario sans fatigue ni oubli
- Il tourne à chaque commit, pas une fois par sprint
- Il documente le comportement attendu
- Il autorise à refactorer sans peur

La pyramide des tests

Unitaires

Une fonction isolée. Très nombreux, très rapides (millisecondes).

Intégration

Plusieurs composants ensemble, souvent avec une base ou une API. Rapides.

End-to-end notre sujet

Un vrai navigateur, une vraie application, un parcours utilisateur complet. Peu nombreux, lents.

La règle pratique

Un test end-to-end coûte cher : il est lent, il dépend de tout, il casse plus souvent. On en écrit donc **peu**, et seulement sur ce qui compte.

Ce qui mérite un test E2E

- Les parcours qui font vivre le produit : connexion, commande, paiement
- Les enchaînements qu'aucun test unitaire ne couvre
- Les régressions qui sont déjà arrivées une fois

Tout tester en end-to-end produit une suite lente que plus personne ne regarde. C'est le premier piège.

Tests manuels et tests automatisés

	TEST MANUEL	TEST AUTOMATISÉ
Coût initial	Faible : on ouvre le navigateur	Élevé, il faut l'écrire
Coût de répétition	Le même à chaque exécution	Quasi nul
Fiabilité	Variable : fatigue, étapes sautées	Identique à chaque passage
Vitesse	Minutes à heures	Secondes à minutes
Exploration	L'humain remarque l'inattendu	Nulle : il vérifie ce qu'on a écrit
Jugement visuel	Naturel	Coûteux à outiller

Ce qu'on automatise

- Ce qui doit **continuer** à marcher
- Ce qui est rejoué à chaque livraison
- Ce dont la panne coûte cher

Ce qu'on laisse à l'humain

- L'exploration et le jugement d'ergonomie
- Un écran qui change encore chaque semaine
- Un cas qui ne sera joué qu'une fois

Le critère : **fréquence de rejeu × coût de la panne × stabilité de l'écran**. En dessous d'un certain seuil, un test automatisé ne se rentabilise pas — et il faudra le maintenir.

DÉFINITION

Qu'est-ce qu'un test end-to-end ?

Un test qui pilote un vrai navigateur pour rejouer le parcours d'un utilisateur réel, du premier clic jusqu'au résultat attendu.

Ce qu'il traverse

- L'interface : DOM, CSS, JavaScript
- Le réseau : appels API, chargements
- Le back-end et ses dépendances
- La base de données

Sa structure, toujours la même

1. **Arranger** : se placer dans l'état de départ
2. **Agir** : reproduire les gestes de l'utilisateur
3. **Vérifier** : affirmer ce qui doit être vrai ensuite

Un test sans étape de vérification n'est pas un test : c'est une macro qui ne dit jamais non.

Playwright en bref

Bibliothèque open source de test end-to-end développée par **Microsoft**, conçue pour les applications web modernes.

Ce qu'elle apporte

- **Trois moteurs** : Chromium, Firefox, WebKit, avec un seul code
- **Auto-wait** intégré : la principale cause d'instabilité disparaît
- **Isolation** : chaque test part d'un navigateur vierge
- **Outillage** : Codegen, Inspector, Trace Viewer, mode UI
- **Parallélisme** natif

Langages disponibles

TypeScript / JavaScript (référence), Python, Java, .NET.

Ce que ça couvre aussi

- Mobile émulé, plusieurs onglets, iframes
- Téléchargements, upload de fichiers
- Interception réseau
- Tests d'API sans navigateur

Nous travaillerons en **TypeScript** : c'est la version de référence, et l'autocomplétion évite beaucoup d'erreurs.

Playwright face aux autres outils

	SELENIUM	CYPRESS	PLAYWRIGHT
Navigateurs	Tous, via WebDriver	Chromium, Firefox, WebKit (partiel)	Chromium, Firefox, WebKit
Attente	À écrire soi-même	Automatique	Automatique
Exécution	Hors du navigateur	Dans le navigateur	Hors du navigateur
Plusieurs onglets / origines	Oui	Limité	Oui
Parallélisme	À outiller	Payant en cloud	Natif et gratuit
Débogage post-mortem	Logs	Rejeu dans l'interface	Trace Viewer

Aucun outil n'est universellement supérieur. Playwright est un bon choix par défaut aujourd'hui pour du web moderne, mais ce qui compte davantage, ce sont les pratiques du module 2.

Browser, context, page

```
Browser           // le processus navigateur, lancé une fois
└─ BrowserContext // une session isolée : cookies, storage, cache
    └─ Page        // un onglet
        └─ Frame   // le document, et ses iframes
```

Pourquoi ça compte

Playwright crée **un contexte neuf par test**. C'est ce qui garantit l'isolation : aucun test n'hérite des cookies ou du localStorage du précédent.

Et c'est peu coûteux, bien moins que relancer un navigateur entier.

La conséquence pratique

Vos tests peuvent tourner **dans n'importe quel ordre**, et **en parallèle**, sans se marcher dessus.

Le corollaire

Un test qui suppose qu'un autre test s'est exécuté avant est cassé par construction. On y revient au module 3.

Installer l'environnement

Une seule commande

```
npm init playwright@latest
```

Elle demande le langage, le dossier des tests, l'ajout d'un workflow GitHub Actions, puis télécharge les navigateurs.

Si les navigateurs manquent

```
npx playwright install  
npx playwright install chromium # un seul  
npx playwright install --with-deps # Linux/CI
```

Ce qui est créé

```
mon-projet/  
├─ tests/  
│   └─ example.spec.ts  
├─ tests-examples/  
├─ playwright.config.ts  
├─ package.json  
└─ .gitignore
```

Derrière un proxy, `npx playwright install` échoue souvent.
Solutions : variable `HTTPS_PROXY`, ou
`PLAYWRIGHT_BROWSERS_PATH` pointant vers une archive locale.

L'assistant d'installation, question par question

1 TypeScript ou JavaScript ?

Répondez TypeScript : l'autocomplétion évite la moitié des erreurs de frappe sur les noms d'API.

2 Nom du dossier de tests

`tests` par défaut. Gardez-le, c'est la convention que tout le monde reconnaîtra.

3 Ajouter un workflow GitHub Actions ?

Oui, même si vous n'êtes pas sur GitHub : le fichier généré est un bon modèle de départ. Nous le reprendrons au module 4.

4 Installer les navigateurs ?

Oui. C'est l'étape longue :

plusieurs centaines de Mo par moteur

— au total un peu plus d'un Go pour les trois, à vérifier selon la version. C'est aussi celle que les proxys bloquent.

Vérifier que tout est en place

```
npx playwright test
```

Le test d'exemple doit passer. Si c'est le cas, votre environnement est bon et vous pouvez oublier l'installation.

Si l'étape des navigateurs échoue

```
set HTTPS_PROXY=http://proxy:8080  
npx playwright install chromium
```

Ou, avec une archive fournie à l'avance :

```
set PLAYWRIGHT_BROWSERS_PATH=D:\pw-browsers
```

playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  retries: 0,
  reporter: 'html',

  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },

  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  ],
});
```

Les réglages qui comptent dès maintenant

<code>testDir</code>	Où sont les tests
<code>baseURL</code>	Permet d'écrire <code>page.goto('/panier')</code>
<code>trace</code>	Enregistre la trace de débogage
<code>screenshot</code>	Capture en cas d'échec
<code>projects</code>	Les navigateurs ciblés
<code>fullyParallel</code>	Tous les tests en parallèle

Nous reviendrons sur `projects`, `retries` et `webServer` au module 4.

Anatomie d'un fichier de test

```
import { test, expect } from '@playwright/test';

test('la page d\'accueil affiche le titre', async ({ page }) => {
  await page.goto('/');

  await expect(page).toHaveTitle(/Boutique/);
  await expect(page.getByRole('heading', { level: 1 })).toHaveText('Nos produits');
});
```

Ligne à ligne

- `test( ... )` déclare un test : un nom lisible, une fonction
- `{ page }` est une **fixture** : Playwright fournit un onglet neuf
- `async` / `await` partout : tout est asynchrone
- `expect( ... )` exprime ce qui doit être vrai

Le `await` oublié est l'erreur numéro un du débutant : le test se termine avant que l'action ait eu lieu, et passe à tort.

Du fichier vide au test complet

1 Importer

Deux objets suffisent : le déclarateur de test, et le vérificateur.

2 Déclarer

Un nom qui décrit le comportement, et une fonction asynchrone.

3 Arriver quelque part

Toujours la première action : se placer dans l'état de départ.

4 Agir

Reproduire les gestes de l'utilisateur, dans l'ordre.

5 Vérifier

Sans cette étape, ce n'est pas un test.

```
import { test, expect } from '@playwright/test';  
// 1  
  
test('ajoute un article au panier', async ({ page }) => {  
  // 2 : le nom dit ce qui est vérifié  
  
    await page.goto('/produits');  
  // 3  
  
    await page.getByRole('button', { name: 'Ajouter' })  
      .first().click();  
  // 4  
  
    await expect(page.getByTestId('badge-panier'))  
      .toHaveText('1');  
  // 5  
});
```

Ces cinq étapes sont les mêmes pour un test de connexion, de paiement ou de recherche. Seul le contenu des étapes 3 à 5 change.

Les fixtures intégrées

Ce qui figure entre accolades dans la signature du test est une **demande** : Playwright prépare la ressource, vous la remet, puis la nettoie.

FIXTURE	CE QUE VOUS RECEVEZ
<code>page</code>	Un onglet neuf. Celle que vous utiliserez dans 95 % des cas.
<code>context</code>	La session isolée qui contient l'onglet : cookies, storage, permissions.
<code>browser</code>	Le navigateur lui-même. Rarement utile directement.
<code>request</code>	Un client HTTP, pour appeler une API sans passer par l'interface.
<code>browserName</code>	Le nom du moteur en cours : utile pour adapter un test.

```
test('deux onglets indépendants',  
  async ({ context }) => {  
    const a = await context.newPage();  
    const b = await context.newPage();  
    await a.goto('/panier');  
    await b.goto('/produits');  
  });
```

Le principe à retenir

Vous ne créez jamais un navigateur à la main, et vous ne le fermez jamais. Vous **demandez** ce dont vous avez besoin, Playwright gère le cycle de vie.
Au module 4, vous écrirez vos propres fixtures sur ce même modèle.

Naviguer, cliquer, saisir

Navigation

```
await page.goto('/produits');  
await page.goBack();  
await page.reload();
```

Clics

```
await page.getByRole('button', { name: 'Ajouter' })  
  .click();  
await page.getByRole('link', { name: 'Panier' }).click();  
await page.getByRole('checkbox').check();
```

Saisie

```
await page.getByLabel('Email').fill('a@exemple.fr');  
await page.getByLabel('Pays').selectOption('France');  
await page.getByLabel('Photo').setInputFiles('chat.png');
```

Clavier

```
await page.getByLabel('Recherche').press('Enter');  
await page.keyboard.type('bonjour');
```

`fill()` remplace tout le contenu du champ. `type()` simule frappe par frappe, utile seulement si l'application réagit à chaque touche.

Le reste des interactions

Souris

```
await loc.hover();
await loc.dblclick();
await loc.click({ button: 'right' });
await loc.click({ modifiers: ['Control'] });
await source.dragTo(cible);
```

Formulaires

```
await loc.check();
await loc.uncheck();
await loc.selectOption(['rouge', 'bleu']);
await loc.clear();
await loc.setInputFiles(['a.pdf', 'b.pdf']);
```

Téléchargement

```
const [fichier] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('link', { name: 'Facture' }).click(),
]);
await fichier.saveAs('./factures/' + fichier.suggestedFilename());
```

Nouvel onglet

```
const [nouvelOnglet] = await Promise.all([
  context.waitForEvent('page'),
  page.getByRole('link', { name: 'Aide' }).click(),
]);
await nouvelOnglet.waitForLoadState();
```

Le motif est toujours le même : on **arme l'attente de l'événement avant** de déclencher l'action, sinon l'événement peut survenir trop tôt.

Lancer les tests

```
npx playwright test           # tout, sans interface
npx playwright test --headed  # navigateur visible
npx playwright test --ui      # mode UI interactif
npx playwright test panier    # par nom de fichier
npx playwright test -g "connexion" # par titre
npx playwright test --project=firefox
npx playwright test --workers=1 # en série
npx playwright test --debug     # pas à pas
npx playwright test --last-failed # les échecs seuls
npx playwright test --only-changed # les fichiers modifiés
```

Voir le rapport

```
npx playwright show-report
```

Le mode UI, à retenir

`--ui` ouvre une interface où l'on voit la liste des tests, chaque étape, le DOM à chaque instant, et où l'on rejoue à volonté. C'est là que l'on passe le plus clair de son temps pendant l'écriture.

Par défaut

Sans option, les tests tournent **en mode headless**, sans fenêtre, et **en parallèle**. C'est le mode de la CI.

La boucle courte

`--last-failed` ne rejoue que ce qui vient d'échouer : sur une suite de deux minutes, c'est deux secondes. C'est le réflexe à prendre quand on corrige.

Quatre façons de lancer, quatre usages

MODE	COMMANDE	CE QUE VOUS VOYEZ	QUAND
Headless	<code>playwright test</code>	Rien, juste le résultat en console	Par défaut, et en CI
Headed	<code>--headed</code>	Le navigateur s'ouvre et joue le test	Comprendre ce qui se passe à l'écran
UI	<code>--ui</code>	Une interface complète : liste, étapes, DOM, rejeu	Pendant l'écriture, en permanence
Debug	<code>--debug</code>	L'Inspector, exécution pas à pas	Quand un test résiste

Le réflexe à prendre

```
npx playwright test --ui
```

Laissez-le ouvert pendant toute la séance d'écriture : il relance automatiquement les tests que vous modifiez.

`--headed` montre le test qui se déroule mais ne permet pas de revenir en arrière. Le mode UI, si. C'est pour cela qu'on lui préfère presque toujours `--ui`.

Lire le rapport HTML

```
npx playwright test  
npx playwright show-report
```

Ce que le rapport contient

- La liste des tests, avec leur durée et leur statut
- Le filtrage par statut, par fichier, par navigateur
- Pour chaque échec : le message, la ligne fautive, la capture d'écran
- La trace téléchargeable, quand elle a été enregistrée
- La distinction entre un échec franc et un test *flaky* passé au second essai

La colonne à surveiller

Un test marqué **flaky** a échoué puis réussi. Le rapport est vert, mais c'est un signal : ce test est le prochain à corriger.

En équipe

Le rapport est un dossier statique. Il se publie tel quel comme artefact de CI, ou sur un serveur interne, pour que chacun consulte l'échec sans relancer les tests.

Lire un échec, ligne par ligne

```
1) panier.spec.ts:12:3 > affiche le total
```

```
Error: expect(locator).toHaveText(expected)
```

```
Locator: getByTestId('total-panier')
```

```
Expected string: "49,00 €"
```

```
Received string: "0,00 €"
```

```
Timeout: 5000ms
```

```
Call log:
```

- expect.toHaveText with timeout 5000ms
- waiting for getByTestId('total-panier')
- locator resolved to <div data-testid="total-panier">0,00 €</div>
- unexpected value "0,00 €"

```
at panier.spec.ts:18:5
```

1 Le fichier et le test

`panier.spec.ts:12` : où chercher.

2 L'assertion qui a échoué

`toHaveText` : on attendait un texte précis.

3 Attendu contre reçu

« 49,00 € » contre « 0,00 € ». L'élément existe, sa valeur est fausse.

4 Le journal d'appels

C'est la partie la plus utile : il dit si l'élément a été trouvé, et ce qu'il contenait.

5 La ligne exacte

Le dernier `at` pointe l'instruction fautive.

Distinction essentielle : « **locator resolved to** » signifie que l'élément a été trouvé. Si cette ligne manque, le problème est le sélecteur, pas la valeur.

Automatiser un parcours utilisateur simple

≈ 60 minutes

En binôme

Application de démonstration

Ce que vous produisez

1. Installer Playwright dans un projet vide et vérifier que le test d'exemple passe
2. Écrire un test qui ouvre la page d'accueil et vérifie son titre
3. Écrire un test qui suit un parcours complet :
 - arriver sur la page d'accueil
 - saisir un terme dans la recherche
 - ouvrir le premier résultat
 - ajouter l'article au panier
 - vérifier que le panier affiche un article
4. Lancer la suite en mode `--ui` puis en mode normal

Points de vigilance

- Un `await` devant chaque action
- Au moins une assertion par test
- Des noms de tests qui décrivent le comportement, pas la technique

Si vous avez fini en avance

Faites échouer volontairement une assertion et observez le message d'erreur produit. Nous nous en resservons au module 3.

Ce qu'il faut retenir

- Un test end-to-end pilote un **vrai navigateur** sur un **parcours réel** : il est précieux, lent, et donc rare.
- L'automatisation prend en charge la **régression** ; l'humain garde **l'exploration**.
- Playwright : trois moteurs, auto-wait, isolation par context, outillage complet.
- Structure invariable : **arranger, agir, vérifier**.
- `npm init playwright@latest` suffit à démarrer ; `playwright.config.ts` centralise les réglages.
- `await` devant chaque action, et une assertion dans chaque test.

Vos tests marchent. Au module 2, on s'assure qu'ils marcheront **encore la semaine prochaine**.

Le quiz en ligne du module 1

Il porte sur ce que le module vient de couvrir : vocabulaire des tests, place de l'end-to-end, anatomie d'un test, exécution et lecture d'un échec.

wytasoft.com/quizz/playwright-tests-ia-module-1-fr

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

ANNEXES DU MODULE 1

AVANCÉ

Interactions et contextes

Tout ce qu'une application réelle ajoute au clic et à la saisie : cadres imbriqués, fenêtres multiples, appareils, documents produits.

Ce que couvrent ces annexes

#	SUJET	#	SUJET
1	Iframes et shadow DOM	4	Réseau : hors ligne, lenteur, limites de l'émulation
2	Fenêtres surgissantes et nouveaux onglets	5	Deux utilisateurs en simultané
3	Émulation : appareil, permissions, position	6	Vérifier un document produit : PDF, CSV, Excel

Ces sujets ne demandent aucune notion nouvelle : ils appliquent les mécanismes du module 1 à des situations que l'on rencontre dès la deuxième application testée.

Atteindre ce qui est encapsulé

Iframe

```
const paiement = page.frameLocator('#cadre-paiement');

await paiement.getByLabel('Numéro de carte')
  .fill('4242424242424242');
await paiement.getByRole('button', { name: 'Payer' })
  .click();
```

Cas typique : un module de paiement hébergé par un prestataire, ou un composant métier intégré depuis une autre application interne.

Iframes imbriquées

```
page.frameLocator('#externe')
  .frameLocator('#interne')
  .getByRole('button', { name: 'Valider' });
```

Shadow DOM

```
// rien de particulier à faire
await page.getByRole('button', { name: 'Valider' }).click();
```

Playwright traverse le shadow DOM ouvert automatiquement. Un sélecteur CSS classique, lui, s'arrête à la frontière du composant.

Ce que ça implique

Les composants web d'un design system interne se testent exactement comme du HTML ordinaire, à condition de cibler par rôle.

Un shadow DOM **fermé** reste inaccessible, à Playwright comme à tout le reste. C'est un choix d'implémentation à faire remonter à l'équipe de développement.

Fenêtres surgissantes et nouveaux onglets

Une fenêtre ouverte par un clic

```
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('button', { name: 'Se connecter avec...' })
    .click(),
]);

await popup.getByLabel('Identifiant').fill('demo');
await popup.getByRole('button', { name: 'Autoriser' }).click();
await popup.waitForEvent('close');

await expect(page.getByText('Connecté')).toBeVisible();
```

Un onglet ouvert par un lien

```
const [onglet] = await Promise.all([
  context.waitForEvent('page'),
  page.getByRole('link', { name: 'Documentation' }).click(),
]);

await onglet.waitForLoadState();
await expect(onglet).toHaveTitle(/Documentation/);
```

Le motif, toujours le même

On **arme l'attente avant** de déclencher l'action. Dans l'ordre inverse, l'événement peut survenir avant qu'on ne l'écoute, et le test attend indéfiniment quelque chose qui a déjà eu lieu.

Boîtes de dialogue natives

```
page.on('dialog', d => d.accept());
await page.getByRole('button',
  { name: 'Supprimer' }).click();
```

`alert`, `confirm` et `prompt` ne sont pas des éléments du DOM : ils se traitent par un écouteur, déclaré avant l'action.

Sans écouteur, Playwright ferme automatiquement ces boîtes. Un test qui « ne fait rien » sur un bouton de suppression vient souvent de là.

Simuler le contexte de l'utilisateur

Appareil et affichage

```
test.use({
  ...devices['iPhone 14'],
  colorScheme: 'dark',
  locale: 'fr-FR',
  timezoneId: 'Europe/Paris',
});
```

Déclaré en tête de fichier, cela s'applique à tous ses tests.
Déclaré dans la config, à tout un projet.

Permissions et position

```
test.use({
  permissions: ['geolocation', 'notifications'],
  geolocation: { latitude: 48.8909, longitude: 2.2381 },
});

await expect(page.getByText('La Défense')).toBeVisible();
```

Pourquoi fixer locale et fuseau

Un agent de CI configuré en anglais affichera « 1,234.50 » là où votre poste affiche « 1 234,50 ». Fixer ces deux réglages rend le test reproductible partout, et supprime une famille entière d'échecs incompréhensibles.

Autres réglages utiles

- `viewport` : une taille d'écran précise
- `reducedMotion` : couper les animations
- `httpCredentials` : authentification HTTP basique
- `extraHTTPHeaders` : un en-tête sur toutes les requêtes

Conditions réseau dégradées

Hors ligne

```
await context.setOffline(true);
await page.getByRole('button', { name: 'Envoyer' }).click();
await expect(page.getByRole('alert'))
  .toHaveText('Vous êtes hors connexion');

await context.setOffline(false);
```

Réseau lent

```
await page.route('**/api/**', async route => {
  await new Promise(r => setTimeout(r, 2000));
  await route.continue();
});

await expect(page.getByRole('progressbar')).toBeVisible();
```

Panne d'un service

```
await page.route('**/api/recommandations',
  route => route.abort());
// la page doit rester utilisable sans ce bloc
```

Ce que ça permet de vérifier

- Les indicateurs de chargement apparaissent, puis disparaissent
- Un service secondaire indisponible ne casse pas la page
- Les messages d'erreur sont compréhensibles
- Les données saisies ne sont pas perdues

L'émulation reproduit la **taille d'écran et les événements tactiles**, pas les performances d'un vrai téléphone ni les particularités de son navigateur. Elle ne remplace pas un test sur appareil réel avant une mise en production critique.

Deux utilisateurs dans le même test

Deux contextes indépendants, ce sont deux sessions complètes, avec leurs cookies et leurs droits propres.

```
test('validation à quatre yeux', async ({ browser }) => {
  const ctxA = await browser.newContext({
    storageState: '.auth/gestionnaire.json' });
  const ctxB = await browser.newContext({
    storageState: '.auth/responsable.json' });
  const a = await ctxA.newPage();
  const b = await ctxB.newPage();
  // A soumet la demande
  await a.goto('/virements/nouveau');
  await a.getByLabel('Montant').fill('15000');
  await a.getByRole('button', { name: 'Soumettre' }).click();
  // B la voit, et la valide
  await b.goto('/validations');
  await b.getByRole('row', { name: /15 000/ })
    .getByRole('button', { name: 'Valider' }).click();
  // A voit le changement d'état
  await a.reload();
  await expect(a.getByTestId('statut')).toHaveText('Validé');

  await ctxA.close();
  await ctxB.close();
});
```

Les parcours que ça débloque

- Circuit de validation, séparation des rôles
- Messagerie ou notification temps réel
- Édition simultanée d'un même document
- Verrou : B ne modifie pas ce que A édite
- Habilitations : ce que A voit, et pas B

Les précautions

- Fermer explicitement les contextes créés à la main
- Plus lent et plus fragile : à réserver aux parcours qui le justifient
- Une fixture dédiée évite de répéter cette mécanique

Souvent le seul moyen de tester une séparation des tâches : par nature, elle ne se vérifie pas avec un seul compte.

Récupérer un document produit

```
test('génère une attestation', async ({ page }) => {
  await page.goto('/documents');
  const [fichier] = await Promise.all([
    page.waitForEvent('download'),
    page.getByRole('button', { name: 'Télécharger' }).click(),
  ]);
  // 1. le nom proposé
  expect(fichier.suggestedFilename())
    .toMatch(/^attestation-\d{4}\.pdf$/);

  // 2. le fichier lui-même
  const chemin = await fichier.path();
  const taille = (await fs.promises.stat(chemin)).size;
  expect(taille).toBeGreaterThan(1000);
});
```

Vérifier qu'un fichier existe et pèse plus de zéro octet est le minimum, et souvent tout ce que font les équipes. Un PDF vide de 40 Ko y passe sans difficulté.

Configurer la conservation

```
// playwright.config.ts
use: { acceptDownloads: true }
```

Actif par défaut. Fichiers supprimés à la fin du contexte, sauf `saveAs()`.

Conserver, ou joindre au rapport

```
await fichier.saveAs('./telechargements/' +
  fichier.suggestedFilename());

await testInfo.attach('attestation', {
  path: await fichier.path(),
  contentType: 'application/pdf',
});
```

Joint au rapport, le document est consultable même pour un échec survenu en CI.

Contrôler le contenu du document

PDF

Playwright ne sait pas lire un PDF : il faut une bibliothèque d'extraction de texte, ajoutée aux dépendances de développement.

```
const donnees = await fs.promises.readFile(chemin);
const { text } = await extrairePdf(donnees);

expect(text).toContain('Attestation de compte');
expect(text).toContain('DUPONT Marie');
expect(text).toMatch(/Solde\s*:\s*1 250,00 €/);
```

CSV

```
const contenu = await fs.promises.readFile(chemin, 'utf8');
const lignes = contenu.trim().split('\n');

expect(lignes[0])
  .toBe('date;libelle;montant');
expect(lignes).toHaveLength(13); // entête + 12 mois
expect(lignes[1]).toMatch(/^2026-01-/);
```

Ce qu'il faut vérifier en priorité

1. **L'identité** : le document concerne bien la bonne personne ou le bon dossier
2. **Les montants** : ils correspondent à ce qu'affichait l'écran
3. **Le nombre de lignes** : rien n'a été tronqué
4. **L'encodage** : les accents sortent correctement

Le piège le plus fréquent

Un export qui perd les accents ou décale les colonnes reste un fichier valide, de la bonne taille. Seule une vérification du contenu le détecte.

Pour un fichier Excel, le principe est identique avec une bibliothèque de lecture de classeur. Vérifiez la feuille attendue, pas seulement la première.

MODULE 2

Écrire des tests web fiables et maintenables

Auto-wait, sélecteurs robustes, assertions, Codegen, et l'IA comme accélérateur.

MODULE 2 · JOUR 1 APRÈS-MIDI

Écrire des tests web fiables et maintenables

DURÉE

≈ 2 h + TP

À LA FIN

Des tests qui survivent aux évolutions de l'application

Ce que couvre ce module

- Utiliser les **assertions** pour valider les comportements attendus
- Comprendre le mécanisme d'**auto-wait** afin de limiter les tests instables
- Mettre en œuvre des **sélecteurs robustes** basés sur les rôles, textes et labels
- Générer automatiquement des scénarios avec **Playwright Codegen**
- Exploiter l'**IA** pour produire et améliorer des scripts de tests

Travaux pratiques : création et amélioration de tests automatisés avec l'aide de l'intelligence artificielle.

Le test instable

Un test *flaky* échoue puis réussit sur le même code, sans qu'on ait rien changé.

Pourquoi c'est grave

- On relance « pour voir », et le test perd toute valeur de signal
- On finit par ignorer les échecs, y compris les vrais
- La suite entière perd la confiance de l'équipe

Une suite dans laquelle on ne croit plus coûte le prix de son entretien sans rien rapporter.

Les trois causes, par fréquence

- | | |
|------------------------|---|
| 1. Timing | Le test agit avant que la page soit prête |
| 2. Sélecteurs | Ils ciblent quelque chose de volatil |
| 3. État partagé | Les tests se marchent dessus |

Playwright traite la première par construction. Les deux autres relèvent de votre manière d'écrire : c'est l'objet de ce module et du suivant.

L'auto-wait

Avant chaque action, Playwright vérifie que l'élément est **réellement actionnable**, et attend le temps qu'il faut. Sans `sleep`.

Les conditions vérifiées avant un clic

Attaché	Présent dans le DOM
Visible	Ni vide, ni <code>display:none</code> , ni <code>visibility:hidden</code>
Stable	Ne bouge plus, animation terminée
Actif	Non <code>disabled</code>
Atteignable	Rien ne le recouvre au point de clic

Ce que l'on n'écrit plus jamais

```
await page.waitForTimeout(3000);
```

Soit c'est trop court et le test devient instable, soit c'est trop long et la suite s'enlise. Les deux à la fois, en général.

Les rares attentes légitimes

```
await page.waitForURL('/confirmation');  
await page.waitForResponse(r =>  
  r.url().includes('/api/panier'));
```

Elles attendent un **événement précis**, pas une durée arbitraire.

Ce que Playwright attend, et ce qu'il n'attend pas

Attendu automatiquement

- L'apparition d'un élément dans le DOM
- La fin d'une animation ou d'une transition
- La levée d'un attribut `disabled`
- La disparition d'un calque qui recouvre la cible
- Le chargement de la page après un `goto`
- La valeur attendue par une assertion web-first

Jamais attendu tout seul

- La fin d'un appel réseau que rien n'affiche à l'écran
- L'écriture effective d'une donnée en base
- Un traitement asynchrone côté serveur, une file de messages
- Un compteur qui se met à jour après un délai applicatif

Dans ces cas, attendez l'événement

```
await page.waitForResponse(r =>
  r.url().includes('/api/commande') && r.ok());

await expect(page.getByTestId('statut'))
  .toHaveText('Validée', { timeout: 15000 });
```

Attendre un **événement** ou une **condition observable** reste correct. Attendre une **durée** ne l'est jamais.

Locator : une description, pas un élément

Ce que fait un locator

Un locator ne capture pas un élément : il décrit **comment le retrouver**. Il est réévalué à chaque usage.

```
const bouton = page.getByRole('button', { name: 'Valider' });

await bouton.click(); // recherché maintenant
await page.reload();
await bouton.click(); // recherché à nouveau, fonctionne
```

C'est pour cela qu'un locator survit à un rendu React ou à un rechargement de page.

Affiner un locator

```
page.getByRole('listitem')
  .filter({ hasText: 'Clavier' })
  .getByRole('button', { name: 'Retirer' });

page.getByRole('row', { name: 'Dupont' })
  .getByRole('cell').nth(2);
```

Quand il y en a plusieurs

```
await expect(page.getByRole('listitem'))
  .toHaveCount(3);

const premier = page.getByRole('listitem').first();
```

Un locator qui correspond à plusieurs éléments provoque une erreur explicite au moment de l'action, pas un choix silencieux.

Choisir un sélecteur robuste

Ciblez ce que l'utilisateur perçoit, pas la manière dont c'est codé.

PRIORITÉ	MÉTHODE	CIBLE	EXEMPLE
1	<code>getByRole</code>	Rôle accessible + nom	<code>getByRole('button', { name: 'Valider' })</code>
2	<code>getByLabel</code>	Champ via son label	<code>getByLabel('Adresse email')</code>
3	<code>getByPlaceholder</code>	Placeholder	<code>getByPlaceholder('Rechercher')</code>
4	<code>getByText</code>	Texte visible	<code>getByText('Commande confirmée')</code>
5	<code>getByTestId</code>	Attribut dédié au test	<code>getByTestId('total-panier')</code>
6	<code>locator('css')</code>	CSS / XPath	En dernier recours seulement

Un sélecteur fondé sur le rôle et le nom accessible teste aussi, indirectement, **l'accessibilité** de votre application. Un bouton introuvable par son rôle est souvent un bouton mal fichu.

Six façons de cibler le même bouton

```
<li class="pd-card css-1a2b3c">
  <h3>Clavier mécanique</h3>
  <span class="price">49,00 €</span>
  <button class="btn btn-primary"
    data-testid="add-42">
    Ajouter au panier
  </button>
</li>
```

Six façons d'atteindre le même élément. La question n'est pas « est-ce que ça marche aujourd'hui », mais « qu'est-ce qui casse ce sélecteur ».

SÉLECTEUR

CASSE QUAND...

```
getByRole('button', { name:
'Ajouter au panier' })
```

Le libellé change — un vrai changement fonctionnel.

```
getById('add-42')
```

On retire l'attribut. Rare : c'est un contrat.

```
getText('Ajouter au panier')
```

Le texte apparaît ailleurs sur la page.

```
locator('.btn-primary')
```

Une autre carte a la même classe. Ambigu.

```
locator('.css-1a2b3c button')
```

Au prochain build : classe générée.

```
locator('li:nth-child(3) button')
```

Dès qu'un produit s'ajoute au catalogue.

Les sélecteurs qui vous trahiront

Fragile

```
page.locator('.btn-primary.mt-4');
page.locator('div > div > span:nth-child(3)');
page.locator('//div[@id="root"]/div[2]/button');
page.locator('.css-1x2y3z'); // CSS-in-JS
page.locator('#\\:r7\\:'); // id généré
```

Ce qui les casse

- Un changement de classe utilitaire
- Une div ajoutée pour la mise en page
- Un build qui renomme les classes
- Un ordre d'éléments qui change

Robuste

```
page.getByRole('button', { name: 'Valider la commande' });
page.getByLabel('Numéro de carte');
page.getByRole('alert');
page.getByTestId('total-panier');
```

Le test décisif

Demandez-vous : « si le designer change la maquette sans changer la fonctionnalité, mon test doit-il casser ? »

Si oui, le sélecteur est mauvais.

Le mode strict

Si un locator correspond à **plusieurs** éléments, Playwright refuse d'agir plutôt que de choisir au hasard.

```
Error: strict mode violation:
getByRole('button') resolved to 4 elements:
  1) <button>Ajouter</button>
  2) <button>Retirer</button>
  3) <button>Valider</button>
  4) <button>Annuler</button>
```

C'est une bonne nouvelle : l'outil vous signale une ambiguïté que Selenium aurait résolue silencieusement, en cliquant sur le premier venu.

Les quatre façons de lever l'ambiguïté

```
// 1. préciser le nom accessible
page.getByRole('button', { name: 'Valider' });

// 2. restreindre à un conteneur
page.getByRole('listitem')
  .filter({ hasText: 'Clavier' })
  .getByRole('button');

// 3. choisir explicitement
page.getByRole('button').first();
page.getByRole('button').nth(2);

// 4. combiner deux critères
page.getByRole('button')
  .and(page.getByTitle('Ajouter'));
```

`.first()` est un aveu : le test ne dit plus *lequel* il vise. Réservez-le aux listes où n'importe quel élément convient.

Quand poser un data-testid

Le `data-testid` n'est pas un aveu d'échec : c'est un **point d'ancrage explicite**, posé volontairement, que personne ne supprimera par accident en refactorant le CSS.

Il se justifie quand...

- L'élément n'a pas de rôle ni de texte stable : un graphique, un conteneur
- Le texte visible change selon la langue ou les données
- Plusieurs éléments identiques doivent être distingués

Le configurer

```
use: { testIdAttribute: 'data-qa' }
```

```
<div data-testid="total-panier">  
  128,40 €  
</div>
```

```
await expect(page.getByTestId('total-panier'))  
  .toHaveText('128,40 €');
```

À négocier avec l'équipe de développement : un testid retiré casse un test. Il fait partie du contrat, au même titre qu'une API.

Les assertions web-first

Les assertions `expect(locator)` **réessaient** jusqu'à ce que la condition soit vraie ou que le délai expire.

```
await expect(page.getByRole('alert'))  
  .toHaveText('Commande enregistrée');
```

Ici, pas besoin d'attendre l'apparition de l'alerte : l'assertion s'en charge.

Sans `await`, l'assertion ne réessaie pas et le test peut passer à tort. C'est l'erreur la plus coûteuse du module.

Les plus utilisées

```
await expect(loc).toBeVisible();  
await expect(loc).toBeHidden();  
await expect(loc).toBeEnabled();  
await expect(loc).toBeChecked();  
await expect(loc).toHaveText('exact');  
await expect(loc).toContainText('partiel');  
await expect(loc).toHaveValue('abc');  
await expect(loc).toHaveCount(3);  
await expect(loc).toHaveAttribute('href', '/panier');  
await expect(page).toHaveURL(/\/confirmation/);  
await expect(page).toHaveTitle(/Boutique/);
```

Les assertions, par famille

Présence

```
toBeVisible()  
toBeHidden()  
toBeAttached()  
toBeInViewport()  
toHaveCount(n)
```

État

```
toBeEnabled()  
toBeDisabled()  
toBeChecked()  
toBeFocused()  
toBeEditable()  
toBeEmpty()
```

Contenu

```
toHaveText('exact')  
toContainText('partiel')  
toHaveValue('abc')  
toHaveValues([ ... ])  
toHaveAttribute('href', '/x')  
toHaveClass(/actif/)  
toHaveCSS('color', 'rgb( ... )')
```

Page

```
toHaveURL(/\/panier/)  
toHaveTitle('Boutique')
```

Négation

```
await expect(loc)  
  .not.toBeVisible();
```

À savoir

`toHaveText` compare le texte **normalisé** : espaces multiples et retours à la ligne sont réduits. Vous n'avez pas à nettoyer vous-même. Passé un tableau, `toHaveText` vérifie une liste entière, dans l'ordre.

Deux familles d'assertions

Web-first : réessaie

```
await expect(page.getByRole('alert'))  
  .toBeVisible();
```

Playwright interroge le DOM en boucle pendant 5 secondes par défaut. C'est ce qu'il faut utiliser.

Générique : évalue une seule fois

```
const texte = await page  
  .getByRole('alert').textContent();  
expect(texte).toBe('Commande enregistrée');
```

La valeur est lue à un instant t. Si l'alerte n'est pas encore là, l'assertion échoue immédiatement.

Règle simple : **prenez le locator à expect**, ne lui passez pas une valeur que vous avez extraite vous-même.

Assertions souples, et conditions qui mettent du temps

Assertions souples

Une assertion normale arrête le test au premier échec. Une assertion **souple** enregistre l'échec et poursuit : vous voyez tous les problèmes d'un écran en une seule exécution.

```
await expect.soft(page.getByTestId('nom'))
  .toHaveText('Dupont');
await expect.soft(page.getByTestId('email'))
  .toHaveText('dupont@test.fr');
await expect.soft(page.getByTestId('statut'))
  .toHaveText('Actif');
// le test échoue à la fin, en listant les trois
```

À réserver aux vérifications d'affichage indépendantes. Si la suite du test n'a aucun sens sans la première assertion, gardez une assertion stricte.

Attendre une condition arbitraire

```
// une valeur qui finit par arriver
await expect.poll(async () => {
  const r = await request.get('/api/statut');
  return (await r.json()).etat;
}, { timeout: 30000 }).toBe('termine');

// un bloc entier qui doit finir par passer
await expect(async () => {
  await page.reload();
  await expect(page.getByRole('alert')).toBeVisible();
}).toPass({ timeout: 20000 });
```

Ces deux outils remplacent les boucles d'attente maison, et surtout les `waitForTimeout` qu'on serait tenté d'écrire ici.

Trois timeouts à ne pas confondre

TIMEOUT	DÉFAUT	PORTE SUR	OÙ LE RÉGLER
Test	30 s	La durée totale d'un test	<code>timeout</code> dans la config
Expect	5 s	Une assertion web-first	<code>expect.timeout</code>
Action	illimité*	Un clic, une saisie	<code>use.actionTimeout</code>

* Borné en pratique par le timeout du test.

Ponctuellement

```
await expect(loc).toBeVisible({ timeout: 15000 });  
test.setTimeout(60000);
```

Augmenter un timeout ne corrige pas un test instable : cela retarde son échec. Cherchez d'abord ce que le test attend réellement.

Quand le texte n'est pas fiable

Les cas problématiques

- Application multilingue : le libellé dépend de la locale du navigateur
- Montants et dates : séparateurs et formats varient
- Textes issus des données : ils changent à chaque jeu de test
- Espaces insécables dans les prix français

Fragile

```
await expect(total)
  .toHaveText('1 234,50 €');
```

Robuste

```
await expect(total)
  .toContainText('1');
await expect(total)
  .toHaveText(/1\s?234,50/);
```

Fixer la locale du test

```
// playwright.config.ts
use: {
  locale: 'fr-FR',
  timezoneId: 'Europe/Paris',
}
```

Le test devient reproductible quelle que soit la machine, y compris un agent de CI configuré en anglais.

La règle

Ciblez par **rôle** et vérifiez par **identifiant de test** dès que le texte visible peut varier pour une raison qui n'est pas fonctionnelle.

Playwright Codegen

```
npx playwright codegen http://localhost:3000

npx playwright codegen --device="iPhone 13" monsite.fr

# enregistrer une session, puis la réutiliser
npx playwright codegen --save-storage=auth.json monsite.fr
npx playwright codegen --load-storage=auth.json monsite.fr

# aligner l'enregistreur sur VOTRE convention
npx playwright codegen --test-id-attribute=data-qa monsite.fr

# générer dans un autre langage
npx playwright codegen --target=python monsite.fr
```

Vous naviguez normalement, Playwright écrit le code, en privilégiant les locators par rôle.

Les boutons utiles de l'enregistreur

- **Record** : enregistre les actions
- **Pick locator** : donne le locator d'un élément pointé
- **Assert visibility / text / value** : insère une assertion

Bon usage

- Découvrir les locators d'une page inconnue
- Poser rapidement le squelette d'un parcours long
- Vérifier qu'un élément est atteignable par son rôle

Mauvais usage

- Livrer le code généré tel quel
- Enregistrer un parcours de 40 étapes d'un bloc

Codegen produit un brouillon linéaire, sans structure. C'est un point de départ.

Deux options qui changent tout

`--test-id-attribute` : si vous utilisez `data-qa`, l'enregistreur génère vos identifiants.

`--target` : onze langages. Codegen sert aussi hors TypeScript.

MODULE 2 · SECONDE PARTIE

L'IA comme accélérateur

Générer, expliquer, améliorer, en gardant la main sur ce qui est vérifié.

Ce que l'IA fait bien, et moins bien

Elle est efficace pour...

- Traduire un scénario écrit en langage naturel en squelette de test
- Convertir du Selenium ou du Cypress vers Playwright
- Expliquer un message d'erreur obscur
- Proposer un locator plus robuste à partir d'un extrait de DOM
- Générer les cas limites auxquels on n'a pas pensé
- Rédiger des jeux de données de test

Elle échoue sur...

- Le DOM réel de votre application, qu'elle ne voit pas — sauf si elle est reliée au navigateur (voir les annexes du module 2)
- Les règles métier implicites de votre domaine
- L'invention d'API Playwright qui n'existent pas
- Les `waitForTimeout` qu'elle produit par habitude
- La pertinence de ce qui est vérifié

L'IA accélère l'écriture. Elle ne décide pas de ce qui mérite d'être testé : c'est votre travail.

Un prompt qui produit du code utilisable

Les cinq ingrédients

1. **Le contexte technique** : Playwright, TypeScript, version
2. **Le DOM réel** : collez l'extrait HTML concerné
3. **Le scénario** : étape par étape, en français
4. **Les contraintes** : locators par rôle, aucune attente fixe
5. **Le format attendu** : un fichier `.spec.ts` complet

Sans le DOM réel, l'IA invente des sélecteurs plausibles mais faux.
C'est l'ingrédient le plus souvent oublié.

Exemple

Tu écris un test Playwright en TypeScript.

DOM de la page de connexion :

```
<form>
  <label for="mail">Adresse email</label>
  <input id="mail" type="email">
  <label for="pwd">Mot de passe</label>
  <input id="pwd" type="password">
  <button type="submit">Se connecter</button>
</form>
```

Scénario :

1. aller sur /connexion
2. saisir un email et un mot de passe valides
3. soumettre
4. vérifier la redirection vers /tableau-de-bord
5. vérifier que le nom de l'utilisateur est affiché

Contraintes :

- locators getByRole et getByLabel uniquement
- assertions web-first, aucun waitForTimeout
- rends un fichier .spec.ts complet

Trois prompts pour le même besoin

Trop vague

```
Écris des tests Playwright  
pour ma page de connexion.
```

Résultat : des sélecteurs inventés, un scénario générique, souvent un `waitForTimeout`. Rien de réutilisable.

Correct

```
Test Playwright TypeScript  
pour une connexion :  
email, mot de passe,  
bouton Se connecter,  
redirection vers /accueil.
```

```
Utilise getByRole  
et getByLabel.
```

Résultat : une structure juste, mais des locators qui ne correspondent pas forcément au vrai DOM.

Exploitable

```
Test Playwright TypeScript.
```

```
DOM réel :  
[coller le formulaire]
```

```
Scénario : 1. ... 2. ...
```

```
Contraintes :  
- getByRole / getByLabel  
- assertions web-first  
- aucun waitForTimeout  
- un fichier .spec.ts complet
```

Résultat : du code qui compile et qui vise les bons éléments. Il reste à relire les assertions.

La différence entre la colonne 2 et la colonne 3 tient à une seule chose : **le DOM réel**. C'est l'information que l'IA ne peut pas deviner.

L'IA sur des tâches précises

Robustifier un locator

Ce locator casse à chaque déploiement :

```
page.locator('.css-1a2b3c  
> div > button')
```

Voici le DOM : [...]

Propose trois locators plus robustes, du plus au moins recommandé, et explique pourquoi.

Étendre la couverture

Voici mon test du parcours de commande : [...]

Quels cas limites ne sont pas couverts ?

Propose 5 tests complémentaires, du plus au moins critique.

Migrer

Convertis ce test Cypress en Playwright TypeScript.

Utilise getByRole plutôt que les sélecteurs CSS d'origine.

Signale ce qui n'a pas d'équivalent direct.

Une demande étroite et documentée donne un résultat exploitable. « Écris-moi des tests pour mon site » n'en donne aucun.

Relire ce que l'IA propose

Checklist de relecture

API réelles ?	Chaque méthode existe-t-elle vraiment dans Playwright ?
Locators	Correspondent-ils au DOM de <i>votre</i> application ?
Attentes fixes	Aucun <code>waitForTimeout</code> résiduel ?
Assertions	Vérifient-elles le comportement métier, ou juste que la page existe ?
Isolation	Le test dépend-il d'un autre test ou d'une donnée créée ailleurs ?
Le test échoue-t-il ?	Cassez volontairement l'application : le test doit devenir rouge.

Le test le plus important

Un test généré qui passe du premier coup n'est pas une bonne nouvelle tant que vous ne l'avez pas vu échouer.

Un test qui ne peut pas échouer ne vérifie rien.

Et côté confidentialité

Vérifiez ce que vous collez dans un assistant : DOM d'une application interne, données réelles, identifiants. Renseignez-vous sur l'outil autorisé chez vous.

Créer et améliorer des tests avec l'IA

≈ 75 minutes

En binôme

Assistant IA autorisé

Déroulé

1. **Sans IA** : reprenez votre test du TP 1 et remplacez tous les sélecteurs par des locators par rôle, label ou texte.
2. **Avec Codegen** : enregistrez un second parcours, puis nettoyez le code produit : nommage, structure, assertions pertinentes.
3. **Avec l'IA** : rédigez un prompt complet (contexte, DOM, scénario, contraintes) pour générer un troisième test.
4. **Relecture** : passez le code généré à la checklist, corrigez ce qui doit l'être.
5. **Preuve** : cassez volontairement l'application et vérifiez que chacun de vos trois tests devient rouge.

Livrable

Trois tests qui passent, dont un généré et corrigé, avec au moins une assertion métier chacun.

Question à trancher en binôme

Sur ce parcours précis, l'IA vous a-t-elle fait gagner du temps, ou en a-t-elle coûté en relecture ? La réponse honnête est intéressante dans les deux sens.

Cinq questions avant la pause

- Q1** Un test échoue une fois sur quatre, toujours sur le même clic. Quelle est la première hypothèse, et laquelle est exclue ?
Sélecteur ambigu ou élément recouvert. Exclue : « il faut allonger le timeout ».
- Q2** Quelle est la différence entre `expect(loc).toHaveText('x')` et `expect(await loc.textContent()).toBe('x')` ?
La première réessaie pendant 5 secondes, la seconde lit une seule fois et échoue si l'élément n'est pas encore là.
- Q3** Dans quel ordre essayez-vous les méthodes de ciblage ?
Rôle, puis label, puis placeholder, puis texte, puis testid. Le CSS en dernier recours.
- Q4** Playwright refuse d'agir en annonçant *strict mode violation*. Que s'est-il passé ?
Le locator correspond à plusieurs éléments. Il faut le préciser, pas le contourner par `.first()`.
- Q5** L'IA vous rend un test qui passe du premier coup. Quelle est l'étape suivante ?
Casser volontairement l'application pour vérifier que le test devient rouge. Un test qui ne peut pas échouer ne vérifie rien.

Ce qu'il faut retenir

- L'instabilité vient du **timing**, des **sélecteurs** et de **l'état partagé**. Playwright règle le premier ; vous réglez les autres.
- L'auto-wait rend `waitForTimeout` inutile, et même nuisible.
- Un locator **décrit** un élément et se réévalue à chaque usage.
- Ciblez ce que l'utilisateur voit : **rôle, label, texte**. Le CSS en dernier recours.
- Passez le **locator** à `expect`, jamais une valeur extraite.
- L'IA écrit vite ; elle ne décide pas de ce qui doit être vérifié. Un test généré se relit, et se voit échouer.

Vos tests sont solides. Demain, on les organise pour qu'ils restent lisibles à cinquante, et on apprend à diagnostiquer un échec en deux minutes.

Le quiz en ligne du module 2

Il porte sur ce que le module vient de couvrir : auto-wait et ses limites, locators et mode strict, assertions, Codegen et tests assistés par IA.

wyta-soft.com/quiz/playwright-tests-ia-module-2-fr

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

Robustesse, vérifications avancées et agents IA

Ce qui rend un test insensible à l'imprévu, et ce que l'on peut vérifier au-delà du texte affiché.

Ce que couvrent ces annexes

#	SUJET
1	Bandeaux et interruptions imprévisibles
2	Maîtriser le temps dans un test
3	Comparaison visuelle

#	SUJET
4	Audits d'accessibilité automatisés
5	Instantanés d'arbre d'accessibilité
6	Matchers personnalisés
7	Le navigateur piloté par un agent IA
8	L'outillage IA livré avec Playwright

Les quatre premiers sujets répondent à la même question que le module 2 : comment obtenir un verdict fiable. Les trois derniers élargissent ce que l'on peut exprimer et ce que l'on peut automatiser — le dernier étant déjà livré dans Playwright.

Gérer ce qui surgit n'importe quand

Un bandeau de consentement, une enquête de satisfaction, une notification de mise à jour : ces éléments apparaissent à un moment non déterministe et recouvrent la cible du clic. C'est une cause classique de tests instables.

La mauvaise réponse

```
if (await banniere.isVisible()) {  
  await banniere.getByRole('button')  
    .click();  
}  
// échoue si le bandeau  
// apparaît après le test
```

La bonne réponse

```
await page.addLocatorHandler(  
  page.getByRole('dialog',  
    { name: 'Cookies' } ),  
  async (d) => {  
    await d.getByRole('button',  
      { name: 'Accepter' } ).click();  
  },  
);
```

Comment ça fonctionne

Le gestionnaire est déclenché **automatiquement**, chaque fois que Playwright constate que l'élément gêne une action, à n'importe quel moment du test. Déclaré une fois dans un `beforeEach` ou une fixture, il couvre toute la suite.

L'alternative, souvent meilleure

Neutraliser la cause : pré-remplir le cookie de consentement dans le `storageState`, ou désactiver la bannière par un paramètre d'environnement de test.

Quand c'est possible, préférez-la : pas de bandeau, pas de gestionnaire, pas de code de contournement à maintenir.

Contrôler l'horloge de la page

```
test('la session expire au bout de 30 minutes',  
  async ({ page }) => {
```

```
    await page.clock.install({  
      time: new Date('2026-09-24T09:00:00') });  
    await page.goto('/tableau-de-bord');
```

```
    // on avance le temps de 31 minutes  
    await page.clock.fastForward('31:00');
```

```
    await expect(page.getByRole('alert'))  
      .toHaveText('Votre session a expiré');  
  });
```

```
    // figer une date pour un affichage déterministe  
    await page.clock.setFixedTime(  
      new Date('2026-01-15T12:00:00'));
```

Ce que ça rend testable

- Expiration de session ou de jeton
- Comptes à rebours et créneaux réservés
- Messages du type « il y a 3 jours »
- Rafraîchissement périodique d'un tableau de bord
- Bannières affichées à une date donnée
- Calculs de dates d'échéance et de valeur

L'horloge contrôlée est celle du **navigateur**. Une expiration décidée côté serveur n'est pas affectée : il faudra la simuler par une route interceptée qui renvoie une réponse d'expiration.

Capturer et comparer

```
test('la fiche produit ne bouge pas',
  async ({ page }) => {
    await page.goto('/produits/42');

    await expect(page).toHaveScreenshot('fiche.png', {
      maxDiffPixelRatio: 0.01,
      mask: [page.getByTestId('date-du-jour')],
      animations: 'disabled',
    });
  });
```

À la première exécution, l'image de référence est créée. Aux suivantes, elle est comparée pixel à pixel, et le rapport affiche les trois vues : attendu, obtenu, différence.

```
# mettre à jour après un changement voulu
npx playwright test --update-snapshots
```

La comparaison peut aussi porter sur un seul composant : `expect(loc).toHaveScreenshot()`. C'est presque toujours préférable à une page entière.

Les trois pièges

- **Le système d'exploitation** : les polices ne sont pas rendues pareil sous Windows et Linux. Les références doivent être générées dans le même environnement que la CI, donc en conteneur.
- **Les contenus mouvants** : dates, avatars, soldes, publicités. Masquez-les.
- **Les animations** : à désactiver, sinon la capture tombe à un instant aléatoire.

Outil puissant mais coûteux à entretenir. Réservez-le à quelques écrans stables et à forte valeur : une page de paiement, un modèle de document, un composant de design system.

Auditer l'accessibilité

```
npm i -D @axe-core/playwright
```

```
import AxeBuilder from '@axe-core/playwright';

test('la page produit est accessible',
  async ({ page }, testInfo) => {
    await page.goto('/produits/42');
    const resultats = await new AxeBuilder({ page })
      .withTags(['wcag2a', 'wcag2aa'])
      .analyze();
    await testInfo.attach('audit-axe', {
      body: JSON.stringify(resultats.violations, null, 2),
      contentType: 'application/json',
    });
    expect(resultats.violations).toEqual([]);
  });
```

Joindre le rapport rend l'échec exploitable : les violations se lisent dans le rapport HTML, sans rien relancer.

Ce que l'audit détecte

- Contrastes insuffisants
- Images sans alternative textuelle
- Champs sans label associé
- Hiérarchie de titres incohérente
- Attributs ARIA mal employés

Ce qu'il ne détecte pas

Une part seulement des critères est automatisable. Navigation au clavier, pertinence des alternatives textuelles et ordre de lecture restent humains.

Lien avec le module 2 : une application qui passe l'audit est une application dont les éléments se ciblent bien par `getByRole`.

Comparer la structure, pas les pixels

La comparaison visuelle échoue au moindre changement de police ou d'espacement. `toMatchAriaSnapshot` compare autre chose : l'**arbre d'accessibilité** — rôles, noms accessibles, imbrication.

```
await expect(carte).toMatchAriaSnapshot(`
  - listitem:
    - heading "Clavier mécanique" [level=3]:
      - link "Clavier mécanique":
        - /url: /produit?id=1
      - text: Périphériques 49,00 €
      - button "Ajouter au panier"
`);
```

Instantané réellement produit par la carte produit de la Boutique interne.

Ce qu'il attrape

- Un bouton qui perd son libellé accessible
- Un titre qui change de niveau
- Un élément qui disparaît de la structure

Ce qu'il n'attrape pas

Une couleur, un alignement, un débordement. Pour cela, la comparaison visuelle reste nécessaire : les deux sont complémentaires.

Deux formes, et un fil rouge

Gabarit dans le test

```
await expect(carte)
  .toMatchAriaSnapshot(`
    - button "Ajouter au panier"
  `);
```

Lisible dans la revue de code : on voit ce qui est attendu sans ouvrir un autre fichier.

Instantané dans un fichier

```
await expect(page.locator('body'))
  .toMatchAriaSnapshot({ name: 'accueil.aria.yml' });
```

```
npx playwright test -u  # met à jour les instantanés
```

Le même arbre, partout

C'est l'arbre que vise `getByRole`, celui qu'auditent les outils d'accessibilité, et celui que lisent les agents IA des annexes 7 et 8. Une application dont l'arbre est propre est donc à la fois **accessible, facile à cibler** et **pilotable par un agent**. Trois bénéfices, un seul effort.

Un instantané trop large devient un test que personne ne relit et que tout le monde régénère avec `-u`. Visez un composant, pas la page entière.

Étendre expect avec vos règles

```
export const expect = baseExpect.extend({

  async aEtreUnMontantTTC(locator: Locator) {
    const texte = await locator.textContent() ?? '';
    const ok = /^\\d{1,3}(\\s\\d{3})*,\\d{2}\\s€$/.test(texte);

    return {
      pass: ok,
      message: () => ok
        ? `Attendu : pas un montant TTC, reçu "${texte}"`
        : `Attendu un montant au format « 1 234,50 € », `
          + `reçu "${texte}"`,
    };
  },
});
```

```
await expect(page.getByTestId('total'))
  .aEtreUnMontantTTC();
```

Quand ça vaut le coup

- La même vérification revient dans dix tests
- Elle exprime une **règle métier**, pas une mécanique technique
- Le message d'échec par défaut serait incompréhensible

Le vrai bénéfice

Le message d'erreur. Comparez :

expected "12.5" to match /^\\d{1,3}.../

contre

Attendu un montant au format « 1 234,50 € », reçu "12.5"

Le second se comprend sans ouvrir le code du test.

Le navigateur piloté par un agent IA

Jusqu'ici, l'IA nous a servi à **écrire du code** que nous exécutons ensuite. Une autre approche a émergé : donner à un agent IA la capacité de **piloter directement le navigateur**, en lui exposant les actions disponibles.

Le principe

1 Un serveur expose le navigateur

Il publie un ensemble d'actions : naviguer, cliquer, saisir, lire l'état de la page.

2 L'agent lit l'état de la page

Non pas une capture d'écran, mais l'**arbre d'accessibilité** : la liste structurée des éléments, avec leur rôle et leur nom. C'est exactement ce que cible `getByRole`.

3 L'agent décide de l'action suivante

Il choisit un élément et une action, en langage naturel traduit en appel outillé.

4 La boucle recommence

Jusqu'à atteindre l'objectif décrit, ou constater l'échec.

Ce que ça ouvre

- Décrire un parcours en français et laisser l'agent le découvrir sur l'application réelle
- Obtenir un test rédigé à partir d'une exécution réellement observée, donc avec les vrais locators
- Explorer une application inconnue pour en dresser la cartographie
- Reproduire un bug décrit par un utilisateur en langage courant

Pourquoi l'arbre d'accessibilité change tout

Un agent qui raisonne sur des captures d'écran devine des coordonnées. Un agent qui lit rôles et noms manipule les mêmes objets que vos tests, et produit donc des locators exploitables.

Corollaire : plus votre application est accessible, plus l'agent est efficace.

Ce que ça change, et ce que ça ne change pas

Les usages qui tiennent

- **Amorcer** un test sur une application qu'on ne connaît pas
- **Explorer** pour trouver les parcours à couvrir
- **Reproduire** un incident décrit en langage courant
- **Diagnostiquer** un échec en laissant l'agent réobserver la page

Les usages qui ne tiennent pas

- **Exécuter la suite de régression** : un agent est lent, coûteux, et non déterministe. Une suite doit donner le même verdict à chaque exécution.
- **Le test qui se répare tout seul** : un test qui s'adapte silencieusement à un changement d'interface ne détecte plus les régressions. C'est l'inverse du but recherché.

Le partage des rôles

Agent IA	Explorer, produire un brouillon, diagnostiquer
Test Playwright	Vérifier, à l'identique, à chaque commit

Le produit de l'agent est du **code relu par un humain**, versionné, exécuté de façon déterministe. Pas un agent branché sur la CI.

Deux réserves. Ce domaine évolue vite : vérifiez l'état des outils avant de vous appuyer dessus. Et un agent qui pilote un navigateur voit tout ce que voit l'utilisateur : à cadrer avec la sécurité avant tout usage sur une application interne.

L'outillage IA livré avec Playwright

Ce qui précède décrivait un principe. Playwright le livre désormais lui-même — c'est déjà dans la version installée sur le projet de démonstration : **trois commandes**, rien de plus à installer.

```
npx playwright mcp
# le serveur MCP : expose le navigateur
# et le lanceur de tests à un agent

npx playwright init-agents --loop claude
# installe les trois agents dans le dépôt

npx playwright init-skills
# installe les skills Playwright
```

`--loop` désigne l'assistant que vous utilisez : `claude`, `codex`, `copilot`, `opencode`, `vscode` — plus `vscode-legacy` pour les versions antérieures.

Ce que MCP change ici

L'agent ne reçoit plus du texte à propos de votre application : il reçoit des **outils** — ouvrir une page, cliquer, lire l'arbre d'accessibilité, lancer un test, le déboguer.

Il agit donc sur **votre** application réelle, pas sur l'idée qu'il s'en fait.

À vérifier avant la session

Ces commandes supposent un assistant installé sur le poste, et une politique DSI qui l'autorise. Le reste de la formation n'en dépend pas.

Une chaîne, pas trois outils

1 planner — explorer et planifier

Il ouvre l'application et la parcourt réellement. Il en tire un plan de test en markdown : chemins nominaux, cas limites, **et cas d'erreur**. Consigne explicite : des scénarios *indépendants, exécutables dans n'importe quel ordre*.

2 generator — rejouer, puis écrire

Pour **chaque** cas du plan, un par un — jamais en parallèle. Il exécute réellement chaque étape dans le navigateur, relit le journal de ce qu'il vient de faire, et écrit alors le test.

3 healer — lancer et réparer

Il exécute la suite, débogue chaque échec, corrige, relance, recommence jusqu'au vert.

Ce que change « rejouer d'abord »

Le code n'est pas deviné à partir d'un DOM collé : il est **dérivé d'une exécution réelle**. Les locators sont ceux que Playwright a effectivement utilisés sur votre page.

L'orchestration est fournie

Un quatrième prompt, `coverage`, enchaîne les trois : plan, puis un appel au générateur par cas, puis le *healer* sur l'ensemble.

Le fichier graine

C'est la pièce qui empêche l'IA d'écrire des tests génériques. Un test de référence — `tests/seed.spec.ts` — que les agents lisent pour apprendre **vos** conventions : vos fixtures, vos Page Objects, votre façon de nommer.

Le plan y renvoie, et chaque test généré en porte la trace :

```
// spec: specs/plan.md
// seed: tests/seed.spec.ts

test.describe('Ajout au panier', () => {
  test('ajoute un article', async ({ page }) => {
    // 1. Ouvrir le catalogue
    await page.goto('/');
    // ...
  });
});
```

Deux lignes qui valent un audit

Chaque test remonte à **l'item du plan** qui l'a motivé et à **la convention** qu'il suit.

« L'IA a écrit nos tests » devient une affirmation vérifiable, fichier par fichier.

Corollaire : la qualité de la graine plafonne celle de tout ce qui sera généré. Si votre test de référence contient un `waitForTimeout`, vous en récolterez partout.

Ce que le *healer* cherche, et ce qu'il s'interdit

Les causes qu'il examine

- Un sélecteur qui ne correspond plus
- Un problème de synchronisation
- Une dépendance à une donnée ou à un environnement
- Un changement de l'application qui invalide l'attente

Ce sont les trois familles du module 3 — **timing, sélecteur, état partagé** — plus la quatrième, celle où le test avait raison.

Les règles qu'il applique

- Jamais `networkidle`, ni d'API dépréciée
- Une erreur à la fois, puis on relance
- Pour une donnée dynamique, un locator par expression régulière

Quand il n'y arrive pas

S'il reste convaincu que le test est juste, il ne le force pas : il le marque `test.fixme()` et écrit en commentaire ce qui se produit à la place.

Autrement dit, il vous rend **un test suspendu et documenté** plutôt qu'un faux vert.

FIN DE LA SECTION : ÉCRIRE DES TESTS WEB FIABLES ET MAINTENABLES

Module suivant

Module 3 : Structurer, maintenir et déboguer une suite de tests.

Puis module 4 : Industrialiser et intégrer dans le cycle de développement.

MODULE 3 · JOUR 2 MATIN

Structurer, maintenir et déboguer une suite de tests

DURÉE

≈ 1 h 30 + TP

À LA FIN

Diagnostiquer un échec en deux minutes, pas en une heure

Ce que couvre ce module

- Organiser les fichiers et scénarios pour faciliter leur **maintenance**
- Comprendre l'**isolation** des tests et les bonnes pratiques de fiabilisation
- Utiliser les outils de débogage : **logs, captures d'écran et Trace Viewer**
- Lire, analyser et améliorer un **scénario existant**
- Corriger des erreurs de tests avec l'**assistance de l'IA**

Travaux pratiques : correction et fiabilisation d'une suite de tests automatisés existante.

Écrire un test coûte une fois. Le maintenir coûte toujours.

Une suite de dix tests se gère sans méthode. À cinquante, les défauts d'organisation deviennent le premier poste de coût.

Les symptômes

- Le même parcours de connexion copié dans quinze fichiers
- Un changement d'écran qui oblige à corriger vingt tests
- Personne ne sait ce que vérifie `test4.spec.ts`
- Des tests désactivés « en attendant », depuis six mois

Les trois leviers

1. **Nommer** : un fichier et un test disent ce qu'ils vérifient
2. **Factoriser** : un parcours répété s'écrit une fois
3. **Isoler** : chaque test crée ce dont il a besoin

Structurer un projet de tests

```
tests/  
├─ auth/  
│   ├── connexion.spec.ts  
│   └── mot-de-passe-oublie.spec.ts  
├─ catalogue/  
│   ├── recherche.spec.ts  
│   └── fiche-produit.spec.ts  
├─ commande/  
│   ├── panier.spec.ts  
│   └── paiement.spec.ts  
├─ pages/  
│   ├── ConnexionPage.ts  
│   └── PanierPage.ts  
├─ fixtures/  
│   └── test-base.ts  
└─ data/  
    └── utilisateurs.ts
```

Le principe

Découpez **par domaine fonctionnel**, pas par type technique.
On cherche « le test du paiement », pas « le test qui utilise un formulaire ».

Nommer un test

```
X test('test 3', ...)  
X test('clic bouton', ...)  
  
✓ test('refuse une carte expirée', ...)  
✓ test('affiche le total TTC après  
    ajout d\'un article', ...)
```

Le nom du test est le message d'erreur que verra la personne d'astreinte à 2 h du matin. Écrivez-le pour elle.

describe et hooks

```
import { test, expect } from '@playwright/test';

test.describe('Panier', () => {

  test.beforeEach(async ({ page }) => {
    await page.goto('/produits');
    await page.getByRole('button', { name: 'Ajouter' })
      .first().click();
  });

  test('affiche le nombre d\'articles', async ({ page }) => {
    await expect(page.getByTestId('badge-panier'))
      .toHaveText('1');
  });

  test('permet de retirer un article', async ({ page }) => {
    await page.getByRole('button', { name: 'Retirer' }).click();
    await expect(page.getByText('Panier vide')).toBeVisible();
  });
});
```

Les hooks disponibles

<code>beforeEach</code>	Avant chaque test, le cas courant
<code>afterEach</code>	Après chaque test, nettoyage
<code>beforeAll</code>	Une fois pour le groupe
<code>afterAll</code>	Une fois à la fin

`beforeAll` partage un état entre les tests du groupe. C'est précisément ce que l'isolation cherche à éviter : réservez-le à des préparations coûteuses et en lecture seule.

Découper un test long avec test.step

```
test('commande complète', async ({ page }) => {

  await test.step('choisir un article', async () => {
    await page.goto('/produits');
    await page.getByRole('button', { name: 'Ajouter' })
      .first().click();
  });

  await test.step('renseigner la livraison', async () => {
    await page.getByRole('link', { name: 'Commander' }).click();
    await page.getByLabel('Adresse').fill('1 rue de la Paix');
    await page.getByRole('button', { name: 'Continuer' }).click();
  });

  await test.step('payer', async () => {
    await page.getByLabel('Numéro de carte').fill('4242424242424242');
    await page.getByRole('button', { name: 'Payer' }).click();
    await expect(page.getByRole('heading'))
      .toHaveText('Commande confirmée');
  });
});
```

Ce que ça change

- Le rapport et la trace affichent les étapes **nommées**, repliables
- En cas d'échec, on sait immédiatement à *quelle* étape ça a lâché
- Le test se lit comme le scénario de recette d'origine
- La durée de chaque étape est mesurée séparément

Quand l'utiliser

Dès qu'un test dépasse une quinzaine de lignes, ou qu'il traverse plusieurs écrans.

`test.step` ne change rien à l'exécution : c'est de la documentation, exploitée par les outils.

L'isolation des tests

Chaque test doit pouvoir tourner seul, dans n'importe quel ordre, en parallèle des autres.

Ce qui casse l'isolation

```
test('crée un compte', ...);  
test('modifie le compte créé', ...);  
test('supprime le compte', ...);
```

- Le test 2 échoue si le test 1 échoue
- Ils ne peuvent pas tourner en parallèle
- On ne peut pas relancer le test 2 seul

Ce qui la préserve

```
test('modifie un compte', async ({ page }) => {  
  const email = `u-${Date.now()}@test.fr`;   
  await creerCompte(email);  
  // ... le test proprement dit  
});
```

- Chaque test crée sa propre donnée
- Des identifiants uniques évitent les collisions
- La création passe par l'API, pas par l'interface

Playwright vous aide déjà : un **BrowserContext** neuf par test, donc ni cookies ni localStorage hérités. Ce qui reste partagé, c'est votre **base de données**.

Annotations et étiquettes

Annotations

```
test.skip('pas encore implémenté', ...);

test.fixme('bug connu, ticket ABC-123', ...);

test('lent mais nécessaire', async ({ page }) => {
  test.slow();           // triple le timeout
});

test('bug ouvert', async ({ page }) => {
  test.fail();           // on attend un échec
});

test.only('en cours d\'écriture', ...);
```

test.only oublié dans un commit désactive toute la suite sans que personne ne s'en aperçoive. **forbidOnly** en CI fait échouer le build : activez-le dès le premier jour.

Étiquettes

```
test('connexion nominale',
  { tag: '@smoke' },
  async ({ page }) => { ... });

test.describe('Paieement',
  { tag: ['@critique', '@lent'] },
  () => { ... });
```

```
npx playwright test --grep @smoke
npx playwright test --grep-invert @lent
```

À quoi ça sert

Lancer les 8 tests critiques à chaque commit et la suite complète la nuit, sans dupliquer aucun fichier ni maintenir de liste à la main.

Le Page Object Model

```
// pages/ConnexionPage.ts
import { Page, Locator } from '@playwright/test';

export class ConnexionPage {
  readonly erreur: Locator;

  constructor(private page: Page) {
    this.erreur = page.getByRole('alert');
  }

  async aller() {
    await this.page.goto('/connexion');
  }

  async seConnecter(email: string, mdp: string) {
    await this.page.getByLabel('Adresse email').fill(email);
    await this.page.getByLabel('Mot de passe').fill(mdp);
    await this.page.getByRole('button', { name: 'Se connecter' })
      .click();
  }
}
```

À l'usage

```
test('refuse un mot de passe erroné',
  async ({ page }) => {
    const connexion = new ConnexionPage(page);
    await connexion.aller();
    await connexion.seConnecter('a@b.fr', 'faux');
    await expect(connexion.erreur)
      .toHaveText('Identifiants incorrects');
  });
```

Le bénéfice

Le libellé du bouton change ? Une seule ligne à corriger, quel que soit le nombre de tests qui s'en servent.

Le piège

Ne mettez pas les assertions métier dans le Page Object : il décrit *comment interagir*, le test décide *ce qui est vrai*. D'où un `Locator` exposé, plutôt qu'un `attendreErreur()`.

Tests pilotés par les données

Quand le même scénario doit être vérifié sur plusieurs jeux de valeurs, on ne copie pas le test : on boucle.

```
const cartes = [
  { num: '4000000000000002', attendu: 'Carte refusée' },
  { num: '4000000000000069', attendu: 'Carte expirée' },
  { num: '4000000000000127', attendu: 'Code incorrect' },
];

for (const cas of cartes) {
  test(`paiement refusé : ${cas.attendu}`, async ({ page }) => {
    await page.goto('/paiement');
    await page.getByLabel('Numéro de carte').fill(cas.num);
    await page.getByRole('button', { name: 'Payer' }).click();
    await expect(page.getByRole('alert')).toHaveText(cas.attendu);
  });
}
```

Pourquoi c'est mieux qu'un test unique en boucle

- Chaque cas apparaît séparément dans le rapport
- Un cas qui échoue n'empêche pas les autres de s'exécuter
- Les cas tournent en parallèle
- On peut relancer un seul cas

La condition

Le titre du test doit être **unique** pour chaque itération : c'est lui qui identifie le cas dans le rapport.

Les données peuvent venir d'un JSON ou d'un CSV lu au chargement du fichier. Elles doivent être lues **hors** du test, au moment de la déclaration.

CHECKLIST

Les bonnes pratiques de fiabilisation

✓ Un test vérifie **un** comportement

✓ Aucun test ne dépend d'un autre

✓ Les données sont créées par le test lui-même

✓ Les identifiants sont uniques

✓ La préparation passe par l'API, pas par l'interface

✓ Locators par rôle, label ou texte

✗ Aucun `waitForTimeout`

✗ Aucun ordre d'exécution supposé

✗ Aucun compte de production

✗ Aucun test désactivé sans ticket associé

✗ Aucune assertion sur une valeur extraite à la main

✗ Aucun sélecteur CSS de classe générée

Cette liste tient sur une page. Affichez-la en revue de code : elle traite la grande majorité des instabilités.

MODULE 3 · SECONDE PARTIE

Déboguer

Un test est rouge. Que s'est-il passé exactement ?

Quatre outils, quatre moments

OUTIL	COMMANDE	QUAND L'UTILISER
Mode headed	<code>--headed</code>	Voir ce que fait le test, en direct
Mode UI	<code>--ui</code>	Pendant l'écriture : rejouer, inspecter chaque étape
Inspector	<code>--debug</code>	Avancer pas à pas, tester un locator à la volée
Trace Viewer	<code>show-trace</code>	Après coup, surtout pour un échec en CI

Le Trace Viewer est le seul qui fonctionne **sur un échec que vous n'avez pas vu se produire**. C'est celui qui change le plus la vie en CI.

L'extension VS Code

Ce qu'elle apporte

- Une **flèche verte** dans la marge pour lancer un test isolé
- Des **points d'arrêt** classiques dans le code du test
- **Pick locator** : cliquer un élément dans le navigateur et récupérer son locator
- **Record new test** : enregistrer directement dans le fichier ouvert
- Le mode **watch** : relance à chaque sauvegarde
- L'affichage des erreurs directement sur la ligne fautive

Le combo le plus rentable

Mode watch activé, navigateur visible à côté de l'éditeur : vous voyez l'effet de chaque modification en une seconde, sans jamais retourner au terminal.

Alternative en ligne de commande

```
npx playwright test --ui
```

Même confort, indépendant de l'éditeur. À privilégier si l'installation d'extensions est restreinte sur vos postes.

Trace Viewer

Activer l'enregistrement

```
// playwright.config.ts
use: {
  trace: 'on-first-retry', // recommandé en CI
  // 'on' | 'off' | 'retain-on-failure'
}
```

Ouvrir une trace

```
npx playwright show-trace trace.zip
```

La trace est jointe au rapport HTML et téléchargeable depuis les artefacts de la CI.

Ce que la trace contient

- La **frise chronologique** de chaque action, avec sa durée
- Une **capture avant / pendant / après** chaque étape
- Le **DOM complet** à chaque instant, inspectable
- Les **appels réseau** émis
- La **console** du navigateur
- Le **code source** aligné sur l'étape sélectionnée

En pratique

Vous cliquez sur l'action qui a échoué, vous voyez la page telle qu'elle était, et vous inspectez le DOM comme dans un navigateur. La cause saute aux yeux en quelques secondes.

Suspendre un test en plein vol

page.pause

```
test('parcours complexe', async ({ page }) => {  
  await page.goto('/produits');  
  await page.getByRole('button', { name: 'Ajouter' }).click();  
  
  await page.pause(); // le test s'arrête ici  
  
  await page.getByRole('link', { name: 'Panier' }).click();  
});
```

L'Inspector s'ouvre. Le navigateur reste dans l'état exact du test : vous pouvez inspecter le DOM, tester des locators, puis reprendre pas à pas.

Dans l'Inspector

Resume	Reprend jusqu'au prochain arrêt
Step over	Exécute une seule action
Pick locator	Donne le locator d'un élément pointé
Explore	Teste un locator et surligne ce qu'il trouve

`page.pause()` est un outil de développement. Oublié dans un commit, il fige la CI jusqu'au timeout du job. Cherchez-le avant de pousser.

Captures, vidéos, journaux

Dans la configuration

```
use: {  
  screenshot: 'only-on-failure',  
  video: 'retain-on-failure',  
  trace: 'on-first-retry',  
}
```

Ponctuellement, dans un test

```
await page.screenshot({  
  path: 'panier.png',  
  fullPage: true  
});  
  
await expect(page).toHaveScreenshot(); // comparaison visuelle
```

Écouter le navigateur

```
page.on('console', msg =>  
  console.log('[console]', msg.text()));  
  
page.on('pageerror', err =>  
  console.log('[erreur JS]', err.message));  
  
page.on('requestfailed', req =>  
  console.log('[réseau KO]', req.url()));
```

Une erreur JavaScript non gérée dans la page explique une bonne part des échecs mystérieux. Ces trois écouteurs la font remonter immédiatement.

Classer un échec en trois questions

1 L'élément a-t-il été trouvé ?

Regardez la ligne *locator resolved to* dans le journal d'appels, ou la capture de la trace. Si non : problème de **sélecteur**. Le sélecteur est ambigu, l'élément est dans une iframe, ou il n'a jamais été rendu.

2 Était-il là au bon moment ?

Comparez les captures « avant » et « après » dans la trace. Si l'élément apparaît juste après l'échec : problème de **timing**. Cherchez l'événement à attendre, pas une durée à rallonger.

3 Le test passe-t-il seul mais échoue-t-il en suite ?

Relancez-le avec `--workers=1`, puis isolé avec `--grep`. S'il ne passe qu'isolé : problème d'**état partagé**. Une donnée créée par un autre test, ou un compte réutilisé.

Ces trois questions couvrent la quasi-totalité des échecs. Posez-les **dans cet ordre** : chacune élimine une famille de causes avant de passer à la suivante.

Lire un test existant et l'améliorer

Avant

```
test('test panier', async ({ page }) => {  
  await page.goto('http://localhost:3000');  
  await page.waitForTimeout(2000);  
  await page.locator('.pd-card:nth-child(1) .btn-add')  
    .click();  
  await page.waitForTimeout(1000);  
  await page.locator('#cart-icon').click();  
  await page.waitForTimeout(2000);  
  const t = await page.locator('.total').textContent();  
  expect(t).toContain('49');  
});
```

Les cinq défauts

- Nom qui ne dit rien
- Trois attentes fixes, 5 secondes perdues
- Sélecteurs CSS positionnels
- URL en dur
- Assertion sur une valeur extraite

Après

```
test('affiche le total après ajout d\'un article',  
  async ({ page }) => {  
    await page.goto('/');  
  
    await page.getByRole('listitem')  
      .filter({ hasText: 'Clavier mécanique' })  
      .getByRole('button', { name: 'Ajouter au panier' })  
      .click();  
  
    await page.getByRole('link', { name: 'Panier' }).click();  
  
    await expect(page.getByTestId('total-panier'))  
      .toHaveText('49,00 €');  
  });
```

Ce qu'on a gagné

- Le nom décrit le comportement vérifié
- Cinq secondes d'attente supprimées
- Le test survit à un changement de mise en page
- L'assertion réessaie toute seule

Corriger un test qui échoue

Ce qu'il faut donner à l'assistant

1. Le **code du test** en entier
2. Le **message d'erreur complet**, sans le tronquer
3. L'extrait de **DOM** concerné, relevé dans la trace
4. Ce que le test est censé vérifier, en une phrase

Ce test Playwright échoue par intermittence, environ une fois sur cinq.

Test : [coller le code]

Erreur : [coller la sortie complète]

DOM au moment de l'échec : [coller l'extrait]

Le test doit vérifier que le total du panier se met à jour après l'ajout d'un article.

Identifie la cause probable, propose un correctif, et explique pourquoi l'échec était intermittent.

Là où l'IA est vraiment bonne

- Décoder un message d'erreur verbeux
- Repérer un `await` manquant
- Distinguer « élément absent » de « élément recouvert »
- Proposer une alternative à un locator ambigu

Là où elle se trompe

Elle a tendance à proposer d'**augmenter le timeout** ou d'ajouter une attente fixe. C'est presque toujours la mauvaise réponse : l'échec est masqué, pas corrigé.

Sans la trace, vous décrivez le symptôme. Avec la trace, vous décrivez la cause, et la réponse change complètement.

Playwright livre désormais un agent qui fait cette boucle lui-même — le *healer*, décrit dans les **annexes du module 2**.

Fiabiliser une suite de tests existante

≈ 75 minutes

En binôme

Suite fournie, volontairement instable

Le point de départ

Une suite de **six tests** vous est fournie. Quatre échouent systématiquement, un une fois sur deux, un seul passe. Aucun n'est bien écrit.

Déroulé

1. **Diagnostiquer** : lancez la suite plusieurs fois, relevez lesquels sont instables
2. **Instrumenter** : activez `trace: 'on'` et ouvrez le Trace Viewer sur chaque échec
3. **Classer** chaque défaut : timing, sélecteur, ou état partagé
4. **Corriger** : sans jamais augmenter un timeout ni ajouter d'attente fixe
5. **Restructurer** : extrayez le parcours de connexion dans un Page Object
6. **Prouver** : dix exécutions consécutives, dix fois vert — `--repeat-each=10`

Contrainte

Interdiction d'utiliser `waitForTimeout` et d'augmenter un timeout. Si vous en avez envie, c'est que la cause n'est pas encore trouvée.

Livrable

La suite corrigée, plus un court tableau : pour chaque test, la cause identifiée et le correctif appliqué.

Aide autorisée

L'IA est autorisée, à condition de lui fournir la trace et de justifier chaque correctif retenu.

Quatre questions

Q1 Un test passe seul et échoue quand toute la suite tourne. Quelle famille de causes visez-vous ?

L'état partagé. Une donnée ou un compte réutilisé entre tests. Relancez avec `--workers=1` pour confirmer.

Q2 Que met-on dans un Page Object, et qu'est-ce qu'on n'y met surtout pas ?

Les interactions et les locators. Pas les assertions métier : c'est le test qui décide ce qui doit être vrai.

Q3 Quel outil de débogage fonctionne sur un échec survenu en CI, la nuit, que personne n'a vu ?

Le Trace Viewer. Il rejoue l'état complet de la page à chaque étape, a posteriori.

Q4 Pourquoi `beforeAll` est-il à manier avec précaution ?

Il partage un état entre les tests du groupe, ce qui casse l'isolation et empêche le parallélisme.

Ce qu'il faut retenir

- Le coût d'une suite est dans sa **maintenance**, pas dans son écriture.
- Découpage **par domaine fonctionnel**, noms de tests qui décrivent un comportement.
- **Isolation** : chaque test crée ses données et ne suppose aucun ordre.
- Le **Page Object** factorise les interactions ; les assertions restent dans le test.
- Quatre outils de débogage : le **Trace Viewer** est celui qui sert en CI.
- Capture, vidéo et trace se règlent une fois dans la config et servent pendant des années.
- L'IA aide à **lire une erreur**, pas à décider du correctif. Méfiez-vous de ses timeouts.

Votre suite est saine. Cet après-midi, on la fait tourner toute seule, à chaque commit.

Le quiz en ligne du module 3

Il porte sur ce que le module vient de couvrir : organisation et isolation, Page Objects, annotations, Trace Viewer et méthode de diagnostic.

wytasoft.com/quizz/playwright-tests-ia-module-3-fr

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

Organisation, données de test et outillage

Ce qui décide de la durée de vie d'une suite : d'où viennent ses données, comment elle est structurée, et qui lit ses résultats.

Ce que couvrent ces annexes

#	SUJET	#	SUJET
1	Fixtures avancées : portée et options	5	Rapports : joindre ses propres pièces
2	Préparation et nettoyage globaux	6	Intégration à l'outillage qualité
3	Stratégie de données de test	7	Tests de composants
4	Données et conformité	8	Migrer depuis Selenium ou Cypress

Les sujets 3 et 4 sont ceux qui font échouer le plus de suites au bout de six mois. Ils méritent d'être traités même quand le temps manque.

Portée d'une fixture

Par test, la valeur par défaut

```
export const test = base.extend({
  panierVide: async ({ page }, use) => {
    await page.goto('/panier/vider');
    await use(page);
  },
});
```

Créée et détruite pour chaque test qui la demande. C'est ce qui garantit l'isolation.

Par worker, pour ce qui coûte cher

```
export const test = base.extend({
  serveurFactice: [async ({}, use) => {
    const s = await demarrerServeur();
    await use(s);
    await s.arreter();
  }, { scope: 'worker' }],
});
```

Créée une fois par processus, partagée par tous les tests qu'il exécute.

Options paramétrables

```
export const test = base.extend({
  role: ['lecteur', { option: true }],

  pageConnectee: async ({ page, role }, use) => {
    await connecter(page, role);
    await use(page);
  },
});
```

```
// dans un fichier de test
test.use({ role: 'administrateur' });

// ou par projet, dans la config
projects: [
  { name: 'admin', use: { role: 'administrateur' } },
  { name: 'client', use: { role: 'lecteur' } },
]
```

Une fixture de portée worker est **partagée** : elle ne doit contenir aucun état modifiable par un test, sous peine de recréer exactement le problème

Préparation et nettoyage globaux

```
// global-setup.ts
import { FullConfig } from '@playwright/test';

export default async function (config: FullConfig) {
  await reinitialiserBaseDeTest();
  await chargerJeuDeDonneesDeReference();
  process.env.LOT = `run-${Date.now()}`;
}
```

```
// playwright.config.ts
export default defineConfig({
  globalSetup: './global-setup.ts',
  globalTeardown: './global-teardown.ts',
});
```

Usages légitimes

- Réinitialiser une base de test
- Charger un jeu de données de référence
- Démarrer un service factice partagé
- Générer un identifiant de campagne commun

Ce qu'il ne faut pas y mettre

La création des données **propres à un test**. Elle appartient au test, sinon vous recréez la dépendance que l'isolation cherche à supprimer.

Pour l'authentification, préférez un projet **setup** avec **dependencies** : il apparaît dans le rapport, produit une trace, et se rejoue seul.

D'où viennent les données de vos tests

C'est la question qui décide de la survie d'une suite. Quatre approches, par ordre croissant de fiabilité.

APPROCHE	COMMENT ÇA MARCHE	CE QUI CASSE
Comptes partagés en dur	Un compte de test connu, réutilisé partout.	Deux tests parallèles modifient le même compte. Quelqu'un change le mot de passe. Le solde n'est plus celui qu'on attend.
Copie de production anonymisée	Un extrait de la base réelle, dépersonnalisé.	L'anonymisation est imparfaite. Le jeu se périmé. Le volume rend les tests lents. Et le risque juridique reste.
Jeu de référence rechargé	Une base fixe, réinitialisée avant chaque campagne.	Interdit le parallélisme si les tests écrivent. Le jeu dérive à mesure qu'on l'enrichit.
Création à la demande	Chaque test crée ce dont il a besoin, par API, avec des identifiants uniques.	Rien, tant que le nettoyage suit. C'est la cible.

En pratique on combine : un socle de référence rechargé pour le décor, et une création à la demande pour tout ce que le test modifie.

Fabriques de données et nettoyage

Une fabrique plutôt qu'un jeu figé

```
// data/clients.ts
export function unClient(surcharge = {}) {
  const n = `${Date.now()}-${Math.random().toString(36).slice(2, 7)}`;
  return {
    email: `client-${n}@test.interne`,
    nom: 'DUPONT',
    prenom: 'Marie',
    statut: 'actif',
    ...surcharge,
  };
}
```

```
const client = unClient({ statut: 'suspendu' });
await request.post('/api/clients', { data: client });
```

Le test ne décrit que **ce qui compte pour lui** ; le reste vient de la fabrique. Un champ obligatoire ajouté au modèle, et une seule ligne change.

Nettoyer par fixture

```
export const test = base.extend({
  client: async ({ request }, use) => {
    const r = await request.post('/api/clients',
      { data: unClient() });
    const cree = await r.json();
    await use(cree); // le test s'exécute
    await request.delete(`/api/clients/${cree.id}`);
  },
});
```

Pourquoi la fixture, et pas afterEach

Il s'exécute **même si le test échoue**, et seulement pour les tests qui ont créé la donnée.

Le filet de sécurité

Un nettoyage périodique en base, sur le nommage et l'ancienneté : un test interrompu laisse toujours des traces.

Données de test et conformité

La règle, sans exception

Aucune donnée personnelle réelle dans un environnement de test, dans un dépôt de code, ou dans une capture d'écran jointe à un rapport.

Un extrait de production « anonymisé » reste une donnée personnelle tant que la réidentification est possible, et elle l'est presque toujours par recoupement.

Ce qui fuit sans qu'on y pense

- Les **captures d'écran** d'échec, publiées en artefact de CI
- Les **traces**, qui contiennent le DOM complet et les réponses réseau
- Les fichiers **HAR** enregistrés, avec leurs jetons
- Les **storageState** versionnés par erreur
- Les prompts envoyés à un **assistant IA** avec un DOM réel collé dedans

Les mesures qui suffisent

1. Des données **synthétiques** générées par fabrique, jamais extraites
2. Un domaine de messagerie dédié et non routable, du type `@test.interne`
3. `.auth/`, `*.har` et les téléchargements dans le `.gitignore`
4. Une **durée de rétention courte** sur les artefacts de CI
5. Le masquage des zones sensibles dans les captures

Ces mesures ne coûtent presque rien si elles sont prises au démarrage du projet. Elles deviennent un chantier une fois la suite en place.

Joindre ses propres pièces au rapport

```
test('la commande est bien enregistrée',
  async ({ page, request }, testInfo) => {

    await passerCommande(page);

    const r = await request.get('/api/commandes/derniere');
    const commande = await r.json();

    // la réponse de l'API, consultable en cas d'échec
    await testInfo.attach('reponse-api', {
      body: JSON.stringify(commande, null, 2),
      contentType: 'application/json',
    });

    // une capture nommée, en plus de celle d'échec
    await testInfo.attach('recapitulatif', {
      body: await page.screenshot(),
      contentType: 'image/png',
    });

    expect(commande.montant).toBe(49);
  });
```

Ce qui vaut la peine d'être joint

- La réponse d'API qui contredit l'affichage
- Le document téléchargé par le test
- Le journal applicatif du moment de l'échec
- Le résultat d'un audit d'accessibilité
- Les données que le test a créées

Pourquoi ça change le quotidien

Un échec en CI à 3 h du matin devient analysable sans reproduire quoi que ce soit. La personne d'astreinte ouvre le rapport et voit exactement ce que le test a vu.

Ne joignez jamais de secret ni de donnée personnelle : ces pièces sont publiées avec le rapport.

Faire remonter les résultats

Plusieurs rapporteurs à la fois

```
reporter: [
  ['list'], // console
  ['html', { open: 'never' }], // humain
  ['junit', { outputFile: 'results.xml' }], // CI
  ['json', { outputFile: 'results.json' }], // maison
  ['./mon-rapporteur.ts'],
]
```

Un rapporteur maison

```
export default class MonRapporteur implements Reporter {
  onTestEnd(test, result) {
    if (result.status === 'failed') {
      envoyerAuCanalQualite({
        titre: test.title,
        fichier: test.location.file,
        duree: result.duration,
        erreur: result.error?.message,
      });
    }
  }
}
```

À quoi ça sert concrètement

- Alimenter un **gestionnaire de campagnes** : chaque test automatisé met à jour son cas de test
- Publier un résumé dans un **canal d'équipe** à chaque exécution nocturne
- Historiser **durée et taux d'échec** pour voir la suite dériver avant qu'elle ne devienne inutilisable

Le format pivot

JUnit XML est lu nativement par la plupart des plateformes de CI et des outils de gestion de tests. C'est le format à produire par défaut, même si personne ne le demande encore.

Le rapport HTML s'adresse à celui qui corrige. Ces formats s'adressent à ceux qui pilotent. Les deux sont nécessaires.

Monter un composant isolément

```
import { test, expect }
  from '@playwright/experimental-ct-react';
import { BoutonPanier } from './BoutonPanier';

test('affiche un état chargé pendant l\'ajout',
  async ({ mount }) => {

    const composant = await mount(
      <BoutonPanier produit={{ id: 42, nom: 'Clavier' }} />
    );

    await composant.getByRole('button').click();
    await expect(composant.getByRole('button'))
      .toBeDisabled();
  });
```

Le composant est monté dans un vrai navigateur, sans le reste de l'application.

	COMPOSANT	END-TO-END
Portée	Un composant	Un parcours
Vitesse	Très rapide	Lent
Réalisme	Partiel	Complet
Détecte	Régression d'un composant	Rupture d'intégration
Fonctionnalité expérimentale : l'API peut évoluer. À évaluer sur un périmètre restreint avant de l'adopter largement.		
Utile surtout si vous maintenez un design system : chaque composant se vérifie sans monter l'application entière.		

Migrer une suite existante

La question n'est presque jamais « comment convertir ce test », mais « dans quel ordre, et jusqu'où ».

1 Ne convertissez pas d'abord

Écrivez **les nouveaux tests** en Playwright, à côté de l'existant : les deux suites cohabitent, ce sont deux commandes.

2 Mesurez l'ancienne suite

Lesquels échouent le plus ? Lesquels n'ont rien détecté depuis un an ? Une partie est à supprimer, pas à migrer.

3 Migrez par valeur, pas par facilité

Les parcours critiques d'abord, les plus instables ensuite : ce sont eux qui rentabilisent l'auto-wait.

4 Ne transposez pas les mauvaises habitudes

Migré ligne à ligne, un test Selenium garde ses attentes fixes et ses XPath. Réécrivez les sélecteurs, sinon rien n'est gagné.

5 Éteignez l'ancienne suite

À une date décidée. Deux suites maintenues en parallèle indéfiniment coûtent plus cher que l'une ou l'autre.

Le gain le plus immédiat

La disparition des attentes explicites : sur une suite Selenium, la principale source d'instabilité et une bonne part du temps d'exécution.

L'IA sur ce chantier

Elle convertit bien la structure, mais transpose les sélecteurs à l'identique, XPath fragiles compris.
Demandez-lui explicitement de proposer des locators par rôle, et de signaler ce qui n'a pas d'équivalent direct.

Table de correspondance

Depuis Selenium

SELENIUM	PLAYWRIGHT
<code>driver.get(url)</code>	<code>page.goto(url)</code>
<code>findElement(By.id)</code>	<code>page.getByTestId</code> ou <code>getByRole</code>
<code>WebDriverWait(...).until(...)</code>	inutile : auto-wait
<code>Thread.sleep()</code>	à supprimer
<code>sendKeys()</code>	<code>fill()</code>
<code>switchTo().frame()</code>	<code>frameLocator()</code>
<code>Actions().moveToElement()</code>	<code>hover()</code>
Grid	<code>projects</code> et <code>--shard</code>

Depuis Cypress

CYPRESS	PLAYWRIGHT
<code>cy.visit(url)</code>	<code>await page.goto(url)</code>
<code>cy.get('.x').click()</code>	<code>await page.getByRole(...).click()</code>
<code>cy.contains('texte')</code>	<code>page.getByText('texte')</code>
<code>cy.intercept()</code>	<code>page.route()</code>
<code>should('be.visible')</code>	<code>await expect(loc).toBeVisible()</code>
<code>cy.fixture()</code>	import direct ou fixture Playwright
chaînage implicite	<code>await</code> explicite partout

Le piège, en venant de Cypress : oublier les `await`. Activez la règle ESLint dédiée dès le premier jour.

MODULE 4

Industrialiser les tests et intégrer l'automatisation

Mocking réseau, fixtures, multi-navigateurs et intégration continue.

MODULE 4 · JOUR 2 APRÈS-MIDI

Industrialiser les tests et intégrer l'automatisation dans le cycle de développement

DURÉE

≈ 2 h + TP

À LA FIN

Une suite qui tourne toute seule à chaque commit

Ce que couvre ce module

- Simuler des appels API grâce au **mocking réseau**
- Mettre en place des **fixtures** pour factoriser et réutiliser les scénarios
- Tester des parcours complets sur **différents navigateurs**
- Exécuter des campagnes de tests **multi-navigateurs**
- Intégrer les tests automatisés dans une **chaîne CI/CD**

Travaux pratiques : mise en place d'une suite complète avec simulation d'API et intégration continue.

Pourquoi simuler le réseau

Sans mock, vos tests dépendent de...

- Un back-end disponible et à jour
- Des données qui ne changent pas sous vos pieds
- Des services tiers : paiement, cartographie, envoi d'e-mails
- Une latence réseau variable

Un test rouge parce qu'un service tiers est en maintenance ne vous apprend rien sur votre application.

Ce que le mock permet

- Provoquer une **erreur 500** à volonté
- Tester un **catalogue vide**, ou dix mille articles
- Simuler une **réponse lente** et vérifier le spinner
- Éviter de créer de vraies commandes chez un prestataire

La juste dose

Gardez quelques tests **de bout en bout réels** sur le parcours critique. Mockez le reste, surtout les cas d'erreur, quasi impossibles à provoquer autrement.

page.route : intercepter une requête

Remplacer une réponse

```
test('affiche le catalogue', async ({ page }) => {
  await page.route('**/api/produits', route =>
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, nom: 'Clavier', prix: 49 },
        { id: 2, nom: 'Souris', prix: 29 },
      ]),
    })
  );

  await page.goto('/produits');
  await expect(page.getByRole('listitem')).toHaveCount(2);
});
```

Les autres gestes

```
// laisser passer
route.continue();

// modifier la requête sortante
route.continue({ headers: { ...h, 'X-Test': '1' } });

// simuler une panne réseau
route.abort();

// simuler une erreur serveur
route.fulfill({ status: 500,
  body: '{"erreur":"indisponible"}' });
```

Le motif `'**/api/produits'` accepte les jokers, ou une expression régulière. Placez la route **avant** le `goto` qui déclenche l'appel.

Tester ce qui est difficile à provoquer

L'erreur serveur

```
await page.route('**/api/paiement',  
  r => r.fulfill({ status: 500 }));  
  
await payer();  
await expect(page.getByRole('alert'))  
  .toHaveText(/réessayer/);
```

L'état vide

```
await page.route('**/api/commandes',  
  r => r.fulfill({  
    status: 200,  
    body: '[]' }));  
  
await expect(page.getByText(  
  'Aucune commande')).toBeVisible();
```

La réponse lente

```
await page.route('**/api/recherche',  
  async r => {  
    await new Promise(  
      res => setTimeout(res, 3000));  
    await r.continue();  
  });  
  
await expect(page.getByRole(  
  'progressbar')).toBeVisible();
```

Ces trois cas représentent une part importante des incidents en production, et ne sont presque jamais testés, parce qu'ils sont impossibles à reproduire à la main.

Créer les données par l'API

Passer par l'interface pour créer un compte, puis un panier, puis une commande, avant même de commencer le test, coûte des dizaines de secondes et multiplie les points de rupture.

```
test('affiche l\'historique des commandes',
  async ({ page, request }) => {

    // préparation : trois appels API, moins d'une seconde
    const u = await request.post('/api/utilisateurs', {
      data: { email: `u-${Date.now()}@test.fr` },
    });
    const { id } = await u.json();
    await request.post('/api/commandes', {
      data: { utilisateur: id, montant: 49 },
    });

    // le test proprement dit
    await page.goto(`/utilisateurs/${id}/commandes`);
    await expect(page.getByRole('listitem')).toHaveCount(1);
  });
```

La règle

Testez par l'interface **ce que vous vérifiez**. Préparez par l'API **tout le reste**.

Le gain

- Un test qui passe de 40 secondes à 6
- Un échec qui pointe le vrai sujet, pas la création du compte
- Des données uniques, donc une isolation réelle

La fixture `request` partage les cookies du contexte : une fois connecté, les appels API le sont aussi.

Enregistrer et rejouer le trafic réseau

1. Enregistrer une fois

```
await page.routeFromHAR('./har/catalogue.har', {
  update: true,      // enregistre le trafic réel
});
await page.goto('/produits');
```

2. Rejouer ensuite

```
await page.routeFromHAR('./har/catalogue.har', {
  update: false,     // sert le contenu enregistré
  notFound: 'abort', // tout le reste est bloqué
});
await page.goto('/produits');
```

Quand c'est le bon outil

- Une page qui fait vingt appels : les mocker à la main serait absurde
- Une API tierce dont vous ne maîtrisez ni le format ni la disponibilité
- Reproduire un bug survenu avec un jeu de données précis

Les limites

- Le HAR fige un instant : il se périmé quand l'API évolue
- Il peut contenir des jetons ou des données personnelles, à nettoyer avant de le versionner

Les fixtures

Une fixture est une ressource que Playwright **prépare avant le test et nettoie après**. Vous en utilisez déjà une : `page`.

```
// fixtures/test-base.ts
import { test as base } from '@playwright/test';
import { ConnexionPage } from '../pages/ConnexionPage';

type Fixtures = {
  connexion: ConnexionPage;
  pageConnectee: Page;
};

export const test = base.extend<Fixtures>({
  connexion: async ({ page }, use) => {
    await use(new ConnexionPage(page));
  },
  pageConnectee: async ({ page }, use) => {
    await page.goto('/connexion');
    await page.getByLabel('Email').fill('demo@test.fr');
    await page.getByLabel('Mot de passe').fill('demo');
    await page.getByRole('button', { name: 'Se connecter' }).click();
    await page.waitForURL('/tableau-de-bord');
    await use(page); // le test s'exécute ici
  },
});
```

À l'usage

```
import { test, expect }
  from '../fixtures/test-base';

test('accède à son profil',
  async ({ pageConnectee }) => {
    await pageConnectee
      .getByRole('link', { name: 'Profil' })
      .click();
    await expect(pageConnectee
      .getByRole('heading'))
      .toHaveText('Mon profil');
  });
```

Fixture ou beforeEach ?

Le `beforeEach` vaut pour tout un groupe. La fixture ne se déclenche que pour les tests qui la **demandent**, et se compose.

Réutiliser une session authentifiée

Se reconnecter par l'interface à chaque test coûte quelques secondes multipliées par le nombre de tests. On le fait **une seule fois**.

```
// auth.setup.ts
import { test as setup } from '@playwright/test';
setup('authentication', async ({ page }) => {
  await page.goto('/connexion');
  await page.getByLabel('Email').fill(process.env.USER!);
  await page.getByLabel('Mot de passe').fill(process.env.PWD!);
  await page.getByRole('button', { name: 'Se connecter' }).click();
  await page.waitForURL('/tableau-de-bord');
  await page.context().storageState({ path: 'auth.json' });
});
```

```
// playwright.config.ts
projects: [
  { name: 'setup', testMatch: /auth\.setup\.ts/ },
  {
    name: 'chromium',
    use: {
      ...devices['Desktop Chrome'],
      storageState: 'auth.json',
    },
    dependencies: ['setup'],
  },
]
```

Le résultat

Chaque test démarre connecté, sans passer par le formulaire. Sur cinquante tests, cela fait souvent plusieurs minutes.

À ne pas oublier

`auth.json` contient une session valide : ajoutez-le au `.gitignore`.

Campagnes multi-navigateurs

```
// playwright.config.ts
projects: [
  { name: 'chromium',
    use: { ...devices['Desktop Chrome'] } },
  { name: 'firefox',
    use: { ...devices['Desktop Firefox'] } },
  { name: 'webkit',
    use: { ...devices['Desktop Safari'] } },
  // et le mobile émulé : Pixel 7, iPhone 14
]
```

```
npx playwright test          # tous
npx playwright test --project=webkit # un seul
```

Adapter un test à un navigateur

```
test('téléchargement du justificatif',
  async ({ page, browserName }) => {
    test.skip(browserName === 'webkit',
      'API de téléchargement non supportée');
    // ...
  });
```

Cadrez les projets

Sans `testMatch`, un projet ramasse **toutes** les specs, y compris celles qui attendent un `storageState` qu'il n'a pas. Donnez à chaque navigateur le périmètre qu'il sait rejouer, pas le dépôt entier.

Une stratégie raisonnable

- Sur chaque commit : **Chromium** seul, pour la vitesse
 - Sur la branche principale ou la nuit : **les trois moteurs**, et le mobile
- Multiplier les navigateurs multiplie le temps d'exécution : pas toujours à payer à chaque push.

Le vrai navigateur de vos utilisateurs

Par défaut, le projet `chromium` lance le **Chromium embarqué** par Playwright — pas le Chrome installé sur le poste. Pour tester ce que vos utilisateurs ont réellement, on précise le **canal** :

```
projects: [  
  { name: 'chrome',  
    use: { ...devices['Desktop Chrome'], channel: 'chrome' } },  
  { name: 'edge',  
    use: { ...devices['Desktop Edge'], channel: 'msedge' } },  
]
```

Canaux : `chrome`, `msedge`, et leurs variantes `-beta`, `-dev`, `-canary`. Le navigateur doit être installé sur la machine — agent de CI compris.

WebKit n'est pas Safari

WebKit est le **moteur** de Safari, pas Safari. C'est la meilleure approximation disponible sous Windows et Linux, et elle attrape l'essentiel des écarts de rendu et d'API.

Si Safari est une cible contractuelle, un contrôle final sur un vrai macOS reste nécessaire.

Le catalogue d'appareils

Plus de deux cents profils dans `devices` : résolution, densité, agent utilisateur, tactile. Le compte bouge d'une version à l'autre. Ils *émulent* un appareil, ils ne le remplacent pas.

Un même test, plusieurs environnements

```
// playwright.config.ts
const CIBLES = {
  local: 'http://localhost:3000',
  recette: 'https://recette.exemple.fr',
  preprod: 'https://preprod.exemple.fr',
};

export default defineConfig({
  use: {
    baseURL: CIBLES[process.env.ENV ?? 'local'],
    ignoreHTTPSErrors: process.env.ENV !== 'preprod',
  },
});
```

```
ENV=recette npx playwright test
```

Tous les `page.goto('/panier')` suivent automatiquement la cible choisie.

Les secrets

Jamais dans le dépôt. Ils viennent des variables d'environnement, alimentées par le coffre de la plateforme de CI.

```
await page.getByLabel('Mot de passe')
  .fill(process.env.PWD_RECETTE!);
```

Ce qui change souvent d'un environnement à l'autre

- L'URL de base et les certificats
- Les comptes de test disponibles
- La possibilité de mocker ou non
- Le volume de données, donc les délais

Parallélisme, workers et sharding

Sur une machine

```
// playwright.config.ts
fullyParallel: true,
workers: process.env.CI ? 4 : undefined,
```

Playwright répartit les fichiers entre plusieurs processus. Sans **workers**, il en ouvre aujourd'hui la moitié des cœurs disponibles — un défaut commode, pas un contrat.

Sur plusieurs machines

```
npx playwright test --shard=1/4
npx playwright test --shard=2/4 # etc.
```

Quatre agents de CI en parallèle, chacun un quart de la suite.

Le prérequis

Le parallélisme n'est possible que si les tests sont **vraiment isolés**. Tout le travail du module 3 sert ici.

Un test qui suppose l'état laissé par un autre échouera de façon aléatoire dès que l'ordre change.

Forcer la série, en dernier recours

```
test.describe.configure({
  mode: 'serial' });
```

À réserver aux cas où l'isolation est réellement impossible, et à documenter.

Sharding et fusion des rapports

1. Découper la suite

```
# quatre agents en parallèle
npx playwright test --shard=1/4 --reporter=blob
npx playwright test --shard=2/4 --reporter=blob
npx playwright test --shard=3/4 --reporter=blob
npx playwright test --shard=4/4 --reporter=blob
```

2. Fusionner en un seul rapport

```
npx playwright merge-reports \
  --reporter=html ./all-blob-reports
```

Sans cette fusion, vous obtenez quatre rapports partiels dont aucun ne donne la vue d'ensemble.

L'ordre de grandeur

1 agent	24 minutes
4 agents	environ 7 minutes
8 agents	environ 4 minutes

Le gain n'est jamais linéaire : l'installation et le démarrage se paient sur chaque agent.

Avec `fullyParallel: true`, Playwright répartit les **tests** entre les shards, pas les fichiers : l'équilibrage reste bon même avec un gros fichier. Sans `fullyParallel`, il répartit les **fichiers**, et leur taille pèse alors directement sur l'équilibrage.

Intégrer dans la chaîne CI/CD

Une suite qui ne tourne que sur le poste de celui qui l'a écrite ne protège personne.

Ce que la CI apporte

- Les tests s'exécutent **à chaque proposition de fusion**, sans que personne y pense
- L'échec est visible **avant** la revue de code
- L'environnement est **identique pour tout le monde**, fini le « chez moi ça marche »
- Les traces et rapports sont conservés comme artefacts

Les quatre étapes, quelle que soit la plateforme

1. Installer Node et les dépendances
2. Installer les navigateurs Playwright
3. Démarrer l'application à tester
4. Lancer les tests et publier le rapport

EXEMPLE

GitHub Actions

```
name: Tests E2E
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: rapport-playwright
          path: playwright-report/
          retention-days: 14
```

Les points qui comptent

`--with-deps` Installe les bibliothèques système Linux dont les navigateurs ont besoin

`npm ci` Installation reproductible à partir du lock

`if: always()` Publie le rapport **même quand les tests échouent** : c'est là qu'il sert

`timeout-minutes` Évite un job bloqué qui consomme des minutes

Démarrer l'application : un service du workflow, ou la clé `webServer` — et Playwright s'en charge lui-même.

EXEMPLE

GitLab CI

```
stages:
  - test

tests-e2e:
  stage: test
  image: mcr.microsoft.com/playwright:v1.63.0-noble
  timeout: 30 minutes
  script:
    - npm ci
    - npx playwright test
  artifacts:
    when: always
    paths:
      - playwright-report/
    expire_in: 2 weeks
    reports:
      junit: results.xml
```

L'image officielle fournit Node, les trois navigateurs et leurs dépendances système : plus de `playwright install`. Les dépendances du projet, elles, s'installent toujours avec `npm ci`.

Démarrer l'application depuis la config

```
// playwright.config.ts
webServer: {
  command: 'npm run start',
  url: 'http://localhost:3000',
  reuseExistingServer: !process.env.CI,
  timeout: 120 * 1000,
}
```

Playwright lance la commande, attend que l'URL réponde, exécute les tests, puis arrête le serveur.

`reuseExistingServer` évite de relancer l'application en local si elle tourne déjà, pratique au quotidien.

La configuration adaptée à la CI

```
export default defineConfig({
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : undefined,
  reporter: process.env.CI
    ? [['html'], ['github'], ['junit',
      { outputFile: 'results.xml' }]]
    : 'list',

  use: {
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
});
```

`forbidOnly`

Fait échouer le build si un `test.only` a été oublié

`retries: 2`

Amortit un incident d'infrastructure. **Ne corrige pas** un test instable : voir la gouvernance, deux slides plus loin

`trace`

Enregistrée seulement au premier réessai, coût maîtrisé

`junit`

Format lu par la plupart des plateformes pour afficher les résultats

`--max-failures`

Arrête après N échecs : inutile de brûler vingt minutes d'agent quand l'application est tombée

Les `retries` masquent l'instabilité : surveillez les tests qui ne passent qu'au second essai. Ce sont vos prochains chantiers, pas des tests verts.

Exécuter dans un conteneur

Le rendu d'une page dépend des polices et des bibliothèques système. Un test qui passe sous Windows peut échouer sur un agent Linux, et inversement.

```
docker run --rm \
  -v $(pwd):/travail -w /travail \
  mcr.microsoft.com/playwright:v1.63.0-noble \
  npx playwright test
```

L'image officielle contient Node, les trois navigateurs et toutes leurs dépendances système. Sa version doit correspondre à celle de votre `package.json`.

Quand ça devient nécessaire

- Vous faites de la comparaison visuelle : les captures dépendent du rendu des polices
- Vos développeurs sont sous Windows et macOS, la CI sous Linux
- Vous voulez reproduire en local un échec qui ne survient qu'en CI

C'est aussi la réponse la plus simple au proxy d'entreprise : l'image contient déjà les navigateurs, il n'y a plus rien à télécharger.

Traiter l'instabilité plutôt que la subir

1 Mesurer

Le rapport HTML distingue déjà les tests *flaky*. Relevez-les à chaque exécution de la branche principale.

2 Rendre visible

Un tableau partagé : le test, sa fréquence d'échec, la date du premier signalement. Sans trace écrite, personne ne s'en occupe.

3 Fixer une règle, et l'outiller

Par exemple : un test instable deux semaines de suite est corrigé, ou retiré de la suite bloquante avec un ticket. `--fail-on-flaky-tests` rend la règle exécutable — la CI échoue dès qu'un retry a rattrapé quelque chose.

4 Interdire la rustine

Augmenter un timeout ou ajouter un `retry` ciblé ne corrige rien : cela déplace le problème et le rend invisible.

Le seuil d'alerte

Au-delà de **2 % de tests instables**, l'équipe commence à relancer par réflexe. À ce moment, la suite a cessé d'être un signal.

Le rôle des retries

`retries: 2` en CI est un amortisseur, pas une solution. Il évite qu'un incident réseau bloque une livraison, à condition que quelqu'un regarde ce qui a été rattrapé.

```
npx playwright test --fail-on-flaky-tests
```

Sur la branche principale, ce drapeau transforme « quelqu'un devrait regarder » en « la CI est rouge ».

Quels tests, à quel moment

DÉCLENCHEUR	CE QU'ON LANCE	BUDGET DE TEMPS
À chaque commit	Unitaires + quelques E2E critiques, Chromium seul	< 5 minutes
Sur une pull request	Toute la suite E2E, Chromium	10 à 15 minutes
Sur la branche principale	Suite complète, trois navigateurs	20 à 30 minutes
La nuit	Tout, plus le mobile émulé et les cas longs	Sans contrainte forte

Le critère décisif : **une suite que l'équipe attend est une suite qu'elle contournera**. Au-delà d'un quart d'heure sur une pull request, découpez ou parallélisez.

Suite complète avec simulation d'API et intégration continue

≈ 70 minutes

En binôme

Synthèse des deux jours

Déroulé

1. **Mocker** : écrivez trois tests qui simulent une réponse API : le cas nominal, la liste vide, l'erreur 500.
2. **Factoriser** : créez une fixture `pageConnectee` et faites-la utiliser par au moins deux tests.
3. **Authentifier une fois** : mettez en place le projet `setup` et le `storageState`, mesurez le temps gagné.
4. **Élargir** : ajoutez les projets Firefox et WebKit, lancez la campagne complète, traitez les écarts.
5. **Automatiser** : écrivez le workflow CI, poussez, vérifiez qu'il tourne et que le rapport est bien publié en artefact.
6. **Prouver** : cassez volontairement l'application, poussez, vérifiez que la CI passe au rouge.

Livrable

Un dépôt avec une suite structurée, des mocks, une fixture d'authentification, trois navigateurs configurés et un pipeline CI vert.

À mesurer

Notez le temps d'exécution avant et après le `storageState`, et avec un puis quatre workers. Ces chiffres vous serviront à argumenter au retour.

Étape finale

La dernière étape est la plus importante : une CI qui ne devient jamais rouge ne protège de rien.

Ce qu'il faut retenir

- `page.route` permet de tester ce qui est **impossible à provoquer à la main** : erreurs serveur, états vides, lenteurs.
- Les **fixtures** factorisent la préparation et se composent entre elles.
- Le **storageState** supprime la reconnexion de chaque test, le gain le plus rentable d'une suite.
- Le **multi-navigateurs** se configure en trois lignes ; sa fréquence est un arbitrage de temps.
- Le **parallélisme** ne fonctionne que sur des tests isolés.
- En CI : `forbidOnly`, `retries`, trace au premier réessai, et le rapport publié **même en cas d'échec**.

Quatre questions

Q1 Vous voulez vérifier le message affiché quand le serveur renvoie une erreur 500. Comment provoquez-vous cette erreur ?

Par `page.route` et `route. fulfill({ status: 500 })`. C'est typiquement le cas impossible à provoquer à la main.

Q2 Quelle différence entre une fixture et un `beforeEach` ?

Le `beforeEach` s'applique à tout un groupe. La fixture n'est déclenchée que par les tests qui la demandent, et se compose avec d'autres.

Q3 Sur une suite de cinquante tests, quel réglage rapporte le plus de temps gagné ?

Le `storageState` : la connexion se fait une fois, pas cinquante.

Q4 Pourquoi publier le rapport de CI avec `if: always()` ?

Sans cela, le rapport n'est publié que quand les tests passent, c'est-à-dire précisément quand on n'en a pas besoin.

Ce que vous emportez

Les quatre acquis

1. **Automatiser** un parcours end-to-end sur une application web moderne
2. **Concevoir** des tests fiables : locators par rôle, assertions web-first, aucune attente fixe
3. **Structurer et déboguer** une suite : isolation, Page Objects, Trace Viewer
4. **Exploiter l'IA** pour générer, corriger et optimiser, en gardant le jugement

Par où commencer lundi

- Choisissez **un** parcours critique et automatisez-le seul
- Branchez-le en CI dès le premier jour, même avec un seul test
- Ajoutez un test à chaque régression constatée en production
- Ne migrez pas une suite existante d'un bloc : commencez par les nouveaux tests

Dix tests fiables et lus par l'équipe valent mieux que deux cents que plus personne ne regarde.

Ressources

Documentation

- **playwright.dev** : documentation officielle, très complète et à jour
- **playwright.dev/docs/best-practices** : la page à lire en priorité
- **playwright.dev/docs/api** : référence des API

Outils du quotidien

- L'extension VS Code officielle : lancer et déboguer un test depuis l'éditeur
- `npx playwright test --ui` : le mode UI, à garder ouvert pendant l'écriture

Ce que nous n'avons pas couvert

- Les tests d'API sans navigateur (`request`)
- La comparaison visuelle (`toHaveScreenshot`)
- Les tests de composants
- Les audits d'accessibilité automatisés

La documentation officielle couvre chacun de ces sujets avec des exemples exécutables : c'est le meilleur point de départ.

Le quiz en ligne du module 4

Il porte sur ce que le module vient de couvrir : mocking réseau, fixtures, session réutilisée, campagnes multi-navigateurs et intégration continue.

wytasoft.com/quizz/playwright-tests-ia-module-4-fr

Il demande un compte

Le quiz est hébergé sur le *Quiz Hub* WyTaSoft. Il faut s'y connecter, ou créer un compte gratuit, avant de commencer : c'est là que les résultats sont conservés.

ANNEXES DU MODULE 4

AVANCÉ

Contraintes d'entreprise et exploitation

Les murs que l'on rencontre en interne, et qui décident si la suite entrera un jour en production.

Ce que couvrent ces annexes

#	SUJET	#	SUJET
1	Tests d'API sans navigateur	5	Proxy, certificats et réseau contraint
2	Authentification multi-rôles	6	Budget de performance
3	SSO et fournisseur d'identité	7	Contrôles de sécurité légers
4	Le second facteur d'authentification	8	Erreurs d'organisation les plus coûteuses

Les sujets 3 à 5 sont les trois blocages qui arrêtent le plus de projets d'automatisation en entreprise, avant même la première ligne de test utile.

Vérifier une API directement

```
test('l\'API refuse une commande sans article',
  async ({ request }) => {

    const r = await request.post('/api/commandes', {
      data: { utilisateur: 12, articles: [] },
    });

    expect(r.status()).toBe(422);
    const corps = await r.json();
    expect(corps.erreur).toBe('Commande vide');
  });
```

Aucun navigateur n'est lancé : ces tests s'exécutent en quelques dizaines de millisecondes.

```
test('la commande apparaît en base',
  async ({ page, request }) => {
    await passerCommande(page);

    const r = await request.get('/api/commandes?dernier=1');
    expect((await r.json())[0].montant).toBe(49);
  });
```

Pourquoi c'est utile dans une suite E2E

- Vérifier les règles métier là où elles sont réellement appliquées
- Couvrir les cas d'erreur sans construire un parcours d'interface
- Contrôler un effet de bord après un parcours : la commande a-t-elle bien été enregistrée, et avec le bon montant ?

Un test d'interface incomplet

Une page peut afficher « Commande confirmée » sans que rien n'ait été enregistré. L'assertion sur l'écran est nécessaire mais pas suffisante : le contrôle par API la complète en une ligne.

Une session par rôle, préparée une fois

```
// auth.setup.ts
const ROLES = ['admin', 'gestionnaire', 'client'];
for (const role of ROLES) {
  setup(`connexion ${role}`, async ({ page }) => {
    await page.goto('/connexion');
    await page.getByLabel('Email')
      .fill(process.env[`MAIL_${role.toUpperCase()}`]!);
    await page.getByLabel('Mot de passe')
      .fill(process.env[`PWD_${role.toUpperCase()}`]!);
    await page.getByRole('button', { name: 'Se connecter' }).click();
    await page.waitForURL('/accueil');
    await page.context()
      .storageState({ path: `.auth/${role}.json` });
  });
}
```

```
// playwright.config.ts
projects: [
  { name: 'setup', testMatch: /auth\.setup\.ts/ },
  {
    name: 'admin',
    testMatch: /\.*\.admin\.spec\.ts/,
    use: { storageState: '.auth/admin.json' },
    dependencies: ['setup'],
  },
  // ... un projet par rôle, sur le même modèle
]
```

Le test devient trivial

```
// suppression.admin.spec.ts
test('un admin peut supprimer',
  async ({ page }) => {
    await page.goto('/utilisateurs');
    await expect(page.getByRole('button',
      { name: 'Supprimer' })).toBeVisible();
  });
```

.auth/ contient des sessions valides : au **.gitignore**, sans exception.

Le mur du SSO

La plupart des applications internes ne portent pas leur propre écran de connexion. L'utilisateur est redirigé vers un fournisseur d'identité, s'authentifie là-bas, et revient avec un jeton.

Pourquoi ça bloque les tests

- Le formulaire de connexion appartient à une **autre application**, souvent une autre équipe, parfois un autre domaine
- Il change sans préavis : votre test casse sans que votre application ait bougé
- Il impose un **second facteur** que le test ne peut pas fournir
- Il détecte parfois l'automatisation et la refuse
- Les comptes nominatifs sont soumis à des politiques de mot de passe et de verrouillage

Conséquence pratique : vous ne pouvez pas vous permettre de traverser ce parcours dans chaque test. Il faut le franchir **une fois**, ou pas du tout.

Le principe directeur

Tester l'authentification de votre application, oui.

Tester le fournisseur d'identité de l'entreprise, non : ce n'est pas votre périmètre, et vous ne pourrez pas le corriger.

Un seul test doit traverser le parcours SSO complet, pour vérifier que l'intégration fonctionne. Tous les autres partent d'une session déjà établie.

Trois stratégies, par ordre de préférence

1 Obtenir le jeton par l'API du fournisseur d'identité

Un compte de service, un appel direct au point de terminaison de jeton, puis injection dans le contexte du navigateur. Aucune interface traversée, donc rien ne casse quand le fournisseur change son écran. La plus robuste, et celle qui demande le plus de négociation.

2 Traverser l'interface une seule fois, puis réutiliser la session

Un projet `setup` qui joue le parcours SSO et enregistre un `storageState` par rôle. Simple, mais dépendant de l'écran du fournisseur. À combiner avec des comptes exemptés de second facteur.

3 Contourner en environnement de recette

Un mode simplifié, activé par configuration, hors production uniquement. Efficace, mais le chemin réel n'est plus testé : gardez au moins un test sur le vrai parcours.

```
// injecter un jeton obtenu par API
await context.addCookies([{
  name: 'session', value: jeton,
  domain: 'appli.interne', path: '/',
}]);

// ou dans le stockage local, selon l'application
await context.addInitScript(t => {
  window.localStorage.setItem('jeton', t);
}, jeton);
```

À négocier avant de commencer

Comptes techniques dédiés, exemptés de second facteur et de rotation, limités à la recette. Demande à porter tôt : elle prend des semaines.

Que faire du second facteur

TYPE DE SECOND FACTEUR	CE QU'ON PEUT FAIRE	VERDICT
Code temporel (TOTP)	Le secret partagé est stocké comme un secret de CI. Une bibliothèque génère le code à six chiffres au moment du test, exactement comme l'application mobile.	Automatisable
Code par courriel	Une boîte de test interrogeable par API, ou un serveur de messagerie factice en recette. Le test lit le message et en extrait le code.	Automatisable
Code par SMS	Nécessite un service de réception programmable. Coûteux, lent, peu fiable.	À éviter
Notification poussée sur mobile	Rien de raisonnable. Il faut une exemption.	Non automatisable
Clé physique, biométrie	Certains navigateurs offrent un authenticateur virtuel pilotable, mais la mise en œuvre est lourde.	Cas par cas

```
// génération d'un code temporel dans le test
const code = genererTotp(process.env.TOTP_SECRET!);
await page.getByLabel('Code de vérification').fill(code);
```

La bonne réponse est presque toujours organisationnelle : un **compte technique exempté** de second facteur, cantonné à la recette, avec des

Réseau d'entreprise contraint

Installer derrière un proxy

```
set HTTPS_PROXY=http://proxy.interne:8080
set NO_PROXY=localhost,127.0.0.1
npx playwright install chromium
```

Cache local de navigateurs

```
set PLAYWRIGHT_BROWSERS_PATH=D:\pw-browsers
// même variable sur les postes et en CI
```

Constitué une fois, distribué en interne : la réponse la plus fiable quand le téléchargement sortant est fermé.

Proxy pour les tests eux-mêmes

```
// playwright.config.ts
use: {
  proxy: {
    server: 'http://proxy.interne:8080',
    bypass: 'localhost,.interne',
  },
}
```

Certificats internes

```
use: { ignoreHTTPSErrors: true }
```

En recette, sur un domaine interne. Jamais sur un environnement exposé.

Certificat client

```
use: {
  clientCertificates: [{
    origin: 'https://appli.interne',
    certPath: './certs/client.pem',
    keyPath: './certs/client.key',
  }],
}
```

La solution qui règle tout d'un coup

L'image Docker officielle contient déjà Node et les trois navigateurs : plus rien à télécharger, donc aucun proxy à configurer.

Certificats et clés ne sont jamais versionnés : ils viennent du coffre à secrets.

Un budget de performance dans les tests

```
test('la page de recherche reste rapide',
  async ({ page }) => {
    await page.goto('/recherche');

    const mesures = await page.evaluate(() => {
      const n = performance.getEntriesByType('navigation')[0];
      return {
        domInteractif: n.domInteractive,
        chargementComplet: n.loadEventEnd,
        transfert: n.transferSize,
      };
    });

    expect(mesures.domInteractif).toBeLessThan(2000);
    expect(mesures.transfert).toBeLessThan(1_500_000);
  });
```

```
// nombre de requêtes émises par la page
let requetes = 0;
page.on('request', () => requetes++);
await page.goto('/recherche');
expect(requetes).toBeLessThan(60);
```

Ce que ça attrape

- Une image de 8 Mo ajoutée sans compression
- Une bibliothèque entière importée pour une fonction
- Une boucle qui déclenche cent appels API
- Une régression de rendu introduite discrètement

Les limites, à énoncer clairement

Un agent de CI a une charge variable : les mesures de **temps** sont bruitées et produisent des tests instables.

Préférez des seuils sur ce qui est **déterministe** : poids transféré, nombre de requêtes, nombre de nœuds du DOM. Gardez les seuils de temps très larges, ou laissez-les à un outil de mesure dédié.

Contrôles de sécurité à portée de test

En-têtes de sécurité

```
test('les en-têtes de sécurité sont posés',
  async ({ page }) => {
    const r = await page.goto('/');
    const h = r!.headers();

    expect(h['strict-transport-security']).toBeTruthy();
    expect(h['x-content-type-options']).toBe('nosniff');
    expect(h['content-security-policy']).toBeTruthy();
    expect(h['x-powered-by']).toBeUndefined();
  });
```

Contrôle d'accès

```
test('un client ne peut pas ouvrir l\'administration',
  async ({ page }) => { // projet « client »
    await page.goto('/administration/utilisateurs');
    await expect(page.getByText('Accès refusé')).toBeVisible();
  });
```

Tester l'accès **refusé** compte autant que l'accès autorisé, et c'est presque toujours oublié.

Pas de secret dans la page

```
const html = await page.content();
expect(html).not.toMatch(/sk_live_|BEGIN PRIVATE KEY/);

const erreurs: string[] = [];
page.on('console', m => {
  if (m.type() === 'error') erreurs.push(m.text());
});
// aucune trace technique en console
expect(erreurs.join('\n')).not.toContain('at Object.');
```

Ce que ce n'est pas

Ces contrôles ne remplacent ni un test d'intrusion ni une analyse de code. Ils vérifient des **régressions** sur des points déjà traités, à un coût nul.

Leur intérêt : une configuration de sécurité perdue lors d'une refonte est détectée le jour même, pas au prochain audit annuel.

Les erreurs d'organisation les plus coûteuses

ERREUR	CE QU'ELLE PRODUIT
Viser un taux de couverture E2E	Une suite lente que l'équipe contourne. La couverture est un objectif de test unitaire.
Confier les tests à une équipe séparée	Des tests écrits après coup, jamais corrigés par ceux qui cassent le code.
Automatiser les scénarios de recette tels quels	Des tests de trente étapes, impossibles à diagnostiquer quand ils échouent.
Laisser la CI rouge « en attendant »	En deux semaines, plus personne ne regarde le résultat.
Repousser le SSO et les données à plus tard	Un prototype qui marche sur un poste et n'entrera jamais en CI.

Ce qui fonctionne

- Le test vit dans le **même dépôt** et la même revue que le code
- Celui qui casse un test le corrige
- Un parcours critique automatisé vaut mieux que vingt parcours secondaires
- La suite bloquante reste sous quinze minutes, quitte à découper
- Un test instable est traité comme un bug, avec un ticket
- L'accès technique est demandé **avant** d'écrire le premier test

La difficulté d'une suite E2E n'est presque jamais technique. Elle est dans la discipline collective qui la maintient vivante, et dans les autorisations obtenues au bon moment.

Où chercher ensuite

Références

- playwright.dev/docs/best-practices : à relire une fois par an
- playwright.dev/docs/api : la référence complète
- playwright.dev/docs/release-notes : les nouveautés, l'outil évolue vite

À explorer selon vos besoins

- Les serveurs de navigateurs distants, pour mutualiser l'exécution
- L'intégration avec votre gestionnaire de campagnes de test
- Les outils de mesure de performance dédiés, en complément des seuils de l'annexe 6

Le conseil qui résume tout

Aucun de ces sujets avancés ne compense une suite mal isolée ou des sélecteurs fragiles.

Si vous ne deviez retenir que trois choses des deux jours : **cibler par rôle, isoler chaque test, brancher la suite en CI dès le premier test.**

Le reste s'ajoute quand le besoin apparaît, pas avant.

Merci

Questions, retours, et évaluation de la formation.

Mehdi Tajmouati · Support de cours transmis en fin de session.

 mehdi.tajmouati@wytasoft.com  www.wytasoft.com  Udemy  Medium

LE FIL PRATIQUE DES DEUX JOURNÉES

La démonstration, les exercices et les corrigés

Une seule application sert toutes les démonstrations, les quatre TP et les 25 exercices. La connaître fait gagner du temps à chaque module.

APPLICATION

Boutique interne

EXERCICES

25, du plus simple au plus difficile

CORRIGÉS

38 tests commentés

PREMIÈRE PARTIE

L'application de démonstration

Ce que c'est, ce qu'elle contient, et pourquoi elle est volontairement imparfaite.

Boutique interne

Un catalogue, un panier, une connexion, des commandes et un circuit de validation. Assez petit pour être compris en cinq minutes, assez réaliste pour poser de vrais problèmes de test.

Démarrer

```
cd demo/tests
npm install
npx playwright install chromium
node lancer.js reference
```

Playwright démarre et arrête l'application lui-même. Pour l'ouvrir à côté pendant que vous cherchez :

```
node ../app/serveur.js
# http://localhost:3000
```

Aucune dépendance

L'application est un seul fichier Node, sans bibliothèque. Elle démarre même si le proxy du client bloque tout téléchargement.

Comptes

<code>client@demo.test</code>	client
<code>responsable@demo.test</code>	responsable
Mot de passe : <code>demo</code>	

Les données de l'application

ID	PRODUIT	PRIX	STOCK
1	Clavier mécanique	49,00 €	12
2	Souris ergonomique	29,00 €	30
3	Écran 27 pouces	289,00 €	4
4	Casque antibruit	159,00 €	0
5	Station d'accueil	189,00 €	7
6	Webcam HD	79,00 €	15
7	Support d'écran	39,00 €	22
8	Tapis de souris XL	19,00 €	40

Le produit 4 est en rupture

Son bouton est **disabled**. C'est le cas de test « ce qui ne doit pas être possible ».

La règle métier

Une commande au-dessus de **1000 €** part « en attente de validation » et doit être validée par un responsable. C'est ce qui rend possible le circuit à quatre yeux, avec deux utilisateurs simultanés.

Ce sur quoi vos tests peuvent s'appuyer

Les identifiants de test, par écran

```
badge-panier  utilisateur      # en-tête
catalogue  prix-<id>  ajouter-<id>  # catalogue
prix  stock  confirmation      # fiche produit
total-panier                                # panier
commande-nouvelle  commande-<id>  # commandes
statut-<id>
validation-<id>                        # validations
```

Un `data-testid` est un **contrat explicite** entre l'application et ses tests. On le pose quand le rôle et le texte ne suffisent pas — pas avant.

Points d'API

```
GET    /api/produits
POST   /api/panier
GET    /api/panier
DELETE /api/panier
POST   /api/connexion
POST   /api/commandes
GET    /api/commandes
GET    /api/export.csv
POST   /api/reset      # remet à zéro
```

À quoi ils servent

À préparer l'état d'un test en une requête plutôt qu'en dix clics, et à vérifier côté serveur ce que l'écran prétend afficher.

`/api/reset` vide les commandes et le panier : de quoi repartir d'un état propre, sans dépendre d'un autre test.

Six défauts volontaires

DÉFAUT	CE QU'IL FAIT APPARAÎTRE	RÉGLAGE
La classe CSS des boutons est régénérée à chaque démarrage	Pourquoi un sélecteur accroché au style casse	automatique
Le badge du panier se met à jour après 350 ms	L'auto-wait, et l'échec d'une lecture faite à la main	DEMO_LATENCE
Un bandeau de consentement surgit entre 60 et 760 ms	Le test instable, et <code>addLocatorHandler</code>	DEMO_BANNIERE
L'export CSV peut être encodé en latin1	Un fichier valide, de la bonne taille, au contenu faux	DEMO_CSV_CASSE
Le catalogue est chargé par le navigateur	L'interception réseau change ce qui s'affiche	automatique
Une commande au-dessus de 1000 € demande validation	Le circuit à quatre yeux, deux contextes	DEMO_SEUIL

Chacun reproduit une situation que vous rencontrerez sur votre propre application. Ils ne sont pas là pour piéger, mais pour que l'exercice ressemble au métier.

Les démonstrations

```
node lancer.js           # la liste
node lancer.js reference  # 43 verts
node lancer.js banniere   # le bandeau
node lancer.js tp         # les 4 TP pas à pas
node lancer.js tp3        # la suite instable
node lancer.js tp3-corrige # son corrigé
node lancer.js csv-casse  # l'export faux
node lancer.js sans-latence # badge immédiat
node lancer.js multi-navigateurs
node lancer.js exercices
node lancer.js solutions
```

Toute option est transmise à Playwright : `node lancer.js reference --ui.`

Pourquoi un lanceur

`VAR=1 npx playwright test` ne fonctionne ni sous PowerShell ni sous cmd. Le lanceur pose les variables lui-même : **la même commande marche partout.**

Deux rouges attendus

`csv-casse` et `sans-latence` se terminent en rouge. C'est le résultat voulu : dans les deux cas, l'échec est la démonstration.

DEUXIÈME PARTIE

Les 25 exercices

Un niveau par module, du premier `goto` au circuit de validation à deux utilisateurs.

Comment travailler

1 Lancez le mode UI

`node lancer.js exercices --ui` relance vos tests à chaque sauvegarde et montre le DOM à chaque étape.

2 Retirez le `.skip`

Chaque exercice est un test désactivé. L'énoncé et ses indices sont juste au-dessus, dans le fichier.

3 Écrivez, relancez

Votre progression se lit au nombre de tests verts. Au départ, 33 tests sont en attente.

4 Faites échouer le test

Cassez volontairement l'assertion. S'il reste vert, il ne vérifie rien.

```
test.skip('e1 · la page d\'accueil...'  
  ↓ retirer le .skip  
test('e1 · la page d\'accueil...')
```

Les corrigés

Dans `exercices-solutions/`, à ouvrir **après** avoir cherché.

```
node lancer.js solutions
```

Un exercice bloqué plus de **dix minutes** ne vous apprend plus rien. Demandez.

Niveau 1 — Premiers pas

#	EXERCICE	CE QUE VOUS Y APPRENEZ
1	La page d'accueil	<code>goto</code> , <code>toHaveTitle</code> , <code>getByRole('heading')</code>
2	Compter et lire	<code>toHaveCount</code> , <code>getByTestId</code> , pourquoi une assertion réessaie
3	La recherche	<code>getByLabel</code> , <code>fill</code> , le cas passant et le cas vide
4	Fiche produit	Naviguer par un clic plutôt que par une URL
5	Ajouter au panier	L'auto-wait sur une mise à jour différée de 350 ms

L'exercice 3 demande **deux** tests : la recherche qui trouve, et celle qui ne trouve rien. Le cas vide est celui qu'on oublie, et celui qui casse en production.

Niveau 2 — Sélecteurs et assertions

#	EXERCICE	CE QUE VOUS Y APPRENEZ
6	Réparer des sélecteurs fragiles	Diagnostiquer trois défauts, puis réécrire
7	Le mode strict	Lever une ambiguïté de quatre façons différentes
8	Produit indisponible	<code>toBeDisabled</code> : l'état plutôt que l'apparence
9	Le panier	Compter des lignes, viser une ligne de tableau
10	Le piège de la lecture manuelle	Pourquoi <code>textContent()</code> échoue là où <code>expect</code> réussit

L'exercice 10 est le seul dont le premier test **doit rester rouge** : il est annoté `test.fail()`. C'est la preuve, exécutable, que lire une valeur à la main n'attend rien.

Niveau 3 — Parcours et structure

#	EXERCICE	CE QUE VOUS Y APPRENEZ
11	Un parcours complet	Enchaîner ajout, retrait, vérification
12	Connexion, les deux cas	Tester aussi l'échec, <code>role="alert"</code>
13	Écrire un Page Object	Séparer les interactions des assertions
14	Un test par jeu de données	Générer des tests depuis un tableau
15	Découper un parcours long	<code>test.step</code> , lisibilité du rapport

L'exercice 14 n'a volontairement **pas de squelette** : générer quatre tests depuis un tableau, c'est précisément la structure que vous devez produire vous-même.

Niveau 4 — Industrialisation

#	EXERCICE	CE QUE VOUS Y APPRENEZ
16	Simuler un catalogue vide	<code>page.route</code> , <code>route. fulfill</code>
17	Simuler une panne	Erreur 500, <code>route.abort</code> , robustesse de la page
18	Préparer par API	<code>page.request</code> , gagner du temps
19	Écrire une fixture	<code>base.extend</code> , préparation et nettoyage
20	Contrôler un document produit	Téléchargement, contenu, encodage

L'exercice 19 non plus n'a pas de squelette : écrire une fixture, c'est écrire la structure. L'exercice 20 est celui qui distingue « le fichier existe » de « le fichier est juste ».

Niveau 5 — Les défis

#	EXERCICE	CE QUE VOUS Y APPRENEZ
21	Deux utilisateurs en simultané	Deux contextes, le circuit de validation
22	Habilitations	Tester l'accès refusé, et le visiteur anonyme
23	Le bandeau qui surgit	<code>addLocatorHandler</code> , ou supprimer la cause
24	Contrôles de sécurité	En-têtes HTTP, absence de secret exposé
25	Défi libre	Choisir vous-même ce qui mérite un test

Pour l'exercice 23, activez le bandeau : `node lancer.js exercices-banniere`. Plusieurs de vos autres tests deviendront intermittents — c'est exactement à cela que ressemble une suite *flaky*.

La méthode de déblocage

1 Ouvrez la trace

Elle est enregistrée pour tous les exercices. Vous y voyez le DOM à l'instant précis de l'échec, pas après.

2 Lisez le journal d'appels

La ligne *locator resolved to* dit si l'élément a été trouvé. Si elle manque, le problème est le **sélecteur**. Si elle est là, c'est la **valeur**.

3 Utilisez Pick locator

Dans le mode UI, il produit le locator correct pour l'élément que vous désignez.

4 Demandez

Au-delà de dix minutes, vous n'apprenez plus, vous vous acharnez.

```
npx playwright show-trace \
  test-results/.../trace.zip
```

Ce qu'il ne faut pas faire

Ajouter un `waitForTimeout`, augmenter un timeout, ou passer au CSS parce que le rôle « ne marche pas ». Ces trois gestes font passer le test **et** conservent le défaut.

TROISIÈME PARTIE

Les corrigés, et ce qu'on en garde

Un corrigé n'est pas une réponse à recopier : c'est une comparaison à faire.

Lire un corrigé

38 tests verts, en une douzaine de secondes

`node lancer.js solutions` couvre les 25 exercices. Chaque corrigé est commenté : la cause, puis le correctif.

Les trois questions à se poser

- Mon test vérifie-t-il un **comportement**, ou juste qu'une page s'affiche ?
- Résisterait-il à un changement de **style** ? de **libellé** ?
- Passerait-il si je le lançais **seul**, ou dépend-il d'un autre test ?

Un corrigé n'est pas la seule réponse

L'exercice 7 lève une ambiguïté de **quatre** façons, toutes correctes. Si votre solution diffère et qu'elle respecte les règles, elle est bonne.

Le cas particulier de l'exercice 10

Son corrigé contient un `test.fail()`. Un test dont l'échec est le résultat attendu reste un test qui documente quelque chose.

Les six règles du jeu

1 **Aucun `waitForTimeout`, aucun timeout augmenté**

Si vous en avez envie, c'est que la cause n'est pas trouvée. Une attente fixe déplace le problème dans le temps, elle ne le règle pas.

2 **Aucun sélecteur CSS ni XPath**

Ciblez par rôle, label ou texte. Le `data-testid` seulement quand rien d'autre ne convient.

3 **Une assertion au moins par test**

Et elle porte sur un comportement, pas sur la présence d'une page.

4 **Chaque test tourne seul**

Dans n'importe quel ordre. Il crée lui-même l'état dont il a besoin.

5 **Un nom de test décrit un comportement**

« ajouter puis retirer met le total à jour », pas « test panier ».

6 **Faites échouer vos tests**

Cassez une assertion ou l'application. Si le test reste vert, il ne vérifie rien.

Bonnes pratiques, en une slide

Cibler

- Le **rôle** d'abord, puis le label, le texte, le `data-testid`
- Le CSS en dernier recours, jamais une classe générée
- Une ambiguïté se lève par `filter` ou `getByRole` imbriqué, pas par `.first()` systématique

Attendre

- Les assertions `expect(locator)` **réessaient** : laissez-les faire
- Lire une valeur à la main n'attend rien
- Un test instable a une cause : elle se trouve dans la trace

Structurer

- Un test = un comportement, et il crée son propre état
- Le Page Object décrit *comment*, le test décide *ce qui doit être vrai*
- Préparer par API ce qui n'est pas l'objet du test

Ces règles ne sont pas propres à Playwright. Elles s'appliquent à n'importe quel outil de test end-to-end — Playwright rend seulement plus facile de les respecter.

Où retrouver quoi

CE QUE VOUS CHERCHEZ	OÙ
Les énoncés détaillés	<code>demo/tests/exercices/README.md</code>
Les 25 squelettes	<code>demo/tests/exercices/</code>
Les corrigés commentés	<code>demo/tests/exercices-solutions/</code>
L'application	<code>demo/app/serveur.js</code>
Les suites de démonstration	<code>demo/tests/specs/</code>
Les 4 TP, étape par étape	<code>demo/tests/tp/</code>
Le TP 3 et son corrigé	<code>demo/tests/tp3-*/</code>
Les workflows CI	<code>demo/ci/</code>

Le premier réflexe

`node lancer.js` sans argument liste les dix-huit scénarios avec leur description. C'est le point d'entrée à retenir.

Pour rejouer chez vous

Le dossier `demo/` est autonome : copiez-le, lancez `npm install`, et tout refonctionne sans réseau une fois les navigateurs installés.

FIN DE LA PARTIE PRATIQUE

À vous

Les exercices se font au fil des deux journées, un niveau après chaque module.

RESTITUTION DES TRAVAUX PRATIQUES

Les quatre TP, pas à pas

De la première ligne au pipeline : chaque solution reconstruite une étape à la fois, et chaque étape exécutable.

`node lancer.js tp` · les fichiers sont dans `demo/tests/tp/`

TRAVAUX PRATIQUES 1

Le parcours utilisateur simple

De l'installation au parcours complet, une étape à la fois. Le fichier grandit ; rien n'est réécrit.

Le point de départ

```
mkdir mon-projet && cd mon-projet
npm init playwright@latest
# TypeScript, dossier tests, pas de CI

npx playwright test
# le test d'exemple doit passer
```

Puis les deux façons de lancer, à montrer toutes les deux :

```
npx playwright test --ui    # pour travailler
npx playwright test         # ce qui tournera en CI
```

Le test d'exemple demande internet

Celui que génère `npm init` vise `playwright.dev`. Derrière un proxy fermé, remplacez son `goto` par l'adresse de l'application locale : c'est exactement l'étape 1.

Épingler la version

```
npm i -D @playwright/test@1.63.0
npx playwright install chromium
```

`npm init` installe toujours la **dernière**. On l'épingle ensuite, pour que toute la salle travaille sur la même.

Ouvrir la page, vérifier le titre

```
import { test, expect } from '@playwright/test';

test('la page d\'accueil affiche le catalogue',
  async ({ page }) => {
    await page.goto('/');

    await expect(page).toHaveTitle(/Boutique interne/);
    await expect(page.getByRole('heading', { level: 1 }))
      .toHaveText('Nos produits');
  });
```

Trois choses, déjà

`test( ... )` déclare un cas, et son nom décrit un **comportement**.

`{ page }` est fourni par Playwright : un onglet neuf, isolé des autres tests.

`await` devant chaque action. Sans exception.

`baseUrl` est dans la configuration : `'/'` suffit, et le test survit à un changement d'adresse.

Saisir un terme et le rechercher

```
await page.goto('/');

await page.getByLabel('Rechercher un produit')
  .fill('clavier');
await page.getByRole('button', { name: 'Rechercher' })
  .click();

await expect(page.getByRole('listitem'))
  .toHaveCount(1);
```

Ce qui s'ajoute

`getByLabel` : le champ est ciblé par son **libellé**, pas par son `id`. Le libellé est ce que perçoit l'utilisateur, et il survit à une refonte du balisage.

Un bouton se désigne par son **rôle** et son texte.

Ne pas s'arrêter au clic

Sans l'assertion finale, le test n'aurait **rien vérifié** : il aurait seulement cliqué.

Ouvrir le premier résultat

```
await page.getByRole('listitem').first()
  .getByRole('link').click();

await expect(page.getByRole('heading', { level: 1 }))
  .toHaveText('Clavier mécanique');
```

On chaîne les locators : d'abord l'élément de liste, puis le lien à l'intérieur.

Par un clic, pas par une URL

Un test qui appelle `goto('/produit?id=1')` ne vérifie plus que le lien existe. On navigue comme l'utilisateur.

Pourquoi `.first()` est légitime ici

Parce que l'étape 2 a établi qu'il n'y a **qu'un** résultat. Sans cette assertion, « le premier » serait une hypothèse jamais contrôlée.

Ajouter au panier, et l'attente qu'on n'écrit pas

Ce qu'il faut écrire

```
await page.getByRole('button',  
  { name: 'Ajouter au panier' }).click();  
  
await expect(page.getByTestId('badge-panier'))  
  .toHaveText('1');
```

Aucune attente. L'assertion **réessaie** jusqu'à la valeur attendue et s'arrête dès qu'elle l'obtient.

Ce que tout le monde écrit d'abord

```
const badge = await page  
  .getByTestId('badge-panier').textContent();  
expect(badge).toBe('1'); // ÉCHOUÉ
```

Le badge est mis à jour **350 ms après** le clic. Cette version lit `'0'`, une seule fois.

La première **attend**. La seconde **photographie**.

Le parcours complet

```
await test.step('rechercher « clavier »', async () => {
  await page.getByLabel('Rechercher un produit').fill('clavier');
  await page.getByRole('button', { name: 'Rechercher' }).click();
  await expect(page.getByRole('listitem')).toHaveCount(1);
});

// ... ouvrir la fiche, ajouter au panier ...

await test.step('vérifier le panier', async () => {
  await page.getByRole('link', { name: /Panier/ }).click();
  await expect(page.getByRole('cell', { name: 'Clavier mécanique' }))
    .toBeVisible();
});
```

Ne pas s'arrêter au badge

Un compteur à **1** et un panier vide seraient parfaitement compatibles avec un bug. On ouvre le panier et on regarde ce qu'il contient.

test.step, facultatif

Les étapes apparaissent nommées et repliables dans le mode UI et dans la trace. C'est le module 3 qui s'annonce.

Trois décisions, et pourquoi

LA DÉCISION	CE QU'ELLE ÉVITE
Cibler par label et par rôle	Un test qui casse à la première refonte du balisage
Vérifier le nombre avant <code>.first()</code>	Un test qui vérifie le mauvais produit sans rien dire
Aucune attente écrite	Un test lent aujourd'hui, instable demain

Rejouer la progression

```
node lancer.js tp1 --ui
```

Les cinq étapes s'exécutent dans l'ordre. Chaque fichier de `demo/tests/tp/` est le test à un moment de sa construction.

Avant de passer à la suite

Cassez une assertion et regardez le message. `toHaveText('2')` sur le badge suffit : le journal d'appels montre la valeur lue à chaque tentative.

TRAVAUX PRATIQUES 2

Créer et améliorer des tests avec l'IA

Sans IA, avec Codegen, avec l'assistant — puis la relecture, qui est la partie qui a de la valeur.

Remplacer les sélecteurs techniques

Ce qu'on écrit par réflexe

```
await page.locator('#q').fill('clavier');  
await page.locator('form.recherche button').click();  
await page.locator('ul.produits li:nth-child(1) a').click();
```

Ces trois lignes **fonctionnent aujourd'hui**. C'est ce qui les rend dangereuses : rien ne signale leur fragilité.

La même chose, ciblée sur l'intention

```
await page.getByLabel('Rechercher un produit')  
  .fill('clavier');  
await page.getByRole('button', { name: 'Rechercher' })  
  .click();  
await page.getByRole('listitem').first()  
  .getByRole('link').click();
```

Le libellé et le rôle sont ce que **perçoit l'utilisateur**. Ils survivent à une refonte du balisage.

Enregistrer, puis nettoyer

Ce que Codegen produit

```
test('test', async ({ page }) => {  
  await page.goto('/');  
  await page.getByLabel('Rechercher un produit').click();  
  await page.getByLabel('Rechercher un produit').fill('clavier');  
  await page.getByRole('button', { name: 'Rechercher' }).click();  
  await page.getByRole('link', { name: 'Clavier mécanique' }).click();  
  await page.getByRole('button', { name: 'Ajouter au panier' }).click();  
  await expect(page).toHaveTitle(/Boutique interne/);  
});
```

Il passe. Et il ne vérifie rien.

Le nom ne décrit rien, le clic inutile est enregistré, et la seule assertion porte sur le **titre de la page** — vrai avant comme après l'ajout.

Ce que Codegen ne peut pas savoir

Il enregistre des **gestes**. Il ignore ce que vous vouliez vérifier. C'est à vous d'ajouter l'assertion et de nommer le comportement.

Le prompt, et ce qui revient toujours

```
Contexte : boutique interne, Playwright 1.63, TypeScript.  
DOM : <li> par produit, bouton « Ajouter au panier »,  
      badge data-testid="badge-panier",  
      total data-testid="total-panier".  
Scénario : ajouter deux produits différents,  
          ouvrir le panier, vérifier le total.  
Contraintes : locators par rôle ou label,  
             aucune attente fixe, une assertion métier au moins.
```

Le **prompt** est le livrable de l'étape, pas le code.

Ce qui revient malgré un bon prompt

Une attente fixe « pour être sûr ».

Une assertion sur la *présence* plutôt que sur la *valeur*.

Un sélecteur CSS là où un rôle existe.

Et ici, l'erreur de fond : **49 € + 29 € = 68 €** dans le test généré. L'IA a additionné sans lire le catalogue.

Quatre corrections, dans cet ordre

CE QU'ON CORRIGE	POURQUOI
L'attente fixe saute	Elle ne réglait rien : les assertions attendent déjà
Le total devient 78,00 €	La seule erreur que la relecture devait attraper
<code>li</code> → <code>getByRole('listitem')</code>	Le vocabulaire de l'utilisateur
Le bouton est nommé	<code>getByRole('button')</code> seul marchait par chance

La checklist, en trois questions

Une attente écrite quelque part ?

L'assertion porte-t-elle sur une **valeur métier** ?

Le test échoue-t-il si je casse l'application ?

Sur ce parcours : la structure en quelques secondes, et **quatre minutes** pour trouver l'erreur de calcul. Le gain est réel, la relecture n'est pas optionnelle.

Casser l'application, voir le rouge

```
await page.route('**/api/produits*', async (route) => {  
  const produits = await (await route.fetch()).json();  
  for (const p of produits)  
    if (p.nom === 'Clavier mécanique') p.prix = 99;  
  await route.fulfill({ json: produits });  
});
```

La régression est **simulée** par une interception réseau : reproductible, sans toucher au serveur. Et cela annonce le module 4.

Attention à ce qu'on croit casser

Le catalogue est rendu par le navigateur, mais le **total du panier est calculé par le serveur**. Fausser `/api/produits` change l'affichage, pas le total.

Un test qui reste vert quand l'application est cassée **ne vérifie rien**. C'est la sixième règle du jeu.

TRAVAUX PRATIQUES 3

Fiabiliser une suite existante

Six tests, quatre défauts, une seule interdiction : ni `waitForTimeout`, ni timeout augmenté.

Le test qui échoue une fois sur deux

```
node lancer.js tp3 --repeat-each=5
```

Ce que dit le journal d'appels, et qui nomme la cause :

```
<div role="dialog" id="banniere-cookies">
  intercepts pointer events
```

Un bandeau apparaît entre **60 et 760 ms** et recouvre la page. Le clic part avant, ou après, selon l'humeur de la machine.

RÉPONSE	VERDICT
<code>waitForTimeout(800)</code>	Interdit, et inefficace : le délai est aléatoire
<code>addLocatorHandler</code>	Correct quand on <i>ne peut pas</i> supprimer la cause
Poser le cookie de consentement	Le meilleur : le bandeau n'apparaît jamais

Le correctif qui n'ajoute rien : il retire

La suite d'origine

```
await page.goto('http://localhost:3000');
await page.waitForTimeout(1500);
await page.locator('...').click();
await page.waitForTimeout(1000);
const badge = await page.locator('.badge').textContent();
expect(badge).toBe('1');
```

Après

```
await page.goto('/');
await page.getByRole('listitem')
  .filter({ hasText: 'Clavier mécanique' })
  .getByRole('button', { name: 'Ajouter au panier' })
  .click();
await expect(page.getByTestId('badge-panier'))
  .toHaveText('1');
```

2,5 secondes d'attente en moins **à chaque exécution**, et un test qui ne dépend plus de la vitesse de la machine.

Trois formes de fragilité

DANS LA SUITE	LE DÉFAUT
<code>.css-a1b2c3</code>	Accroché au style — régénéré à chaque démarrage, donc déjà mort
<code>li:nth-child(1)</code>	Positionnel — casse si l'ordre du catalogue change
<code>input[type=email]</code>	Technique — alors qu'un label existe

Remonter d'un cran

Du **balisage** vers l'**intention**. Aucun des trois ne dit ce que l'utilisateur voit.

```
page.getByLabel('Adresse email')
page.getByRole('button', { name: 'Se connecter' })
```

Et une assertion qui ne prouvait rien

Le test de connexion vérifiait que l'URL contenait « localhost ». C'était vrai même quand la connexion échouait.

Le défaut le plus cher à diagnostiquer

```
let totalAttendu = 0;           // variable de module

test('test 1', ... { totalAttendu = 49; });

test('test 2', async ({ page }) => {
  await page.goto('/panier');
  const total = await page.locator('[data-testid=total-panier]')
    .textContent();
  expect(total).toBe(`${totalAttendu},00 €`);
});
```

Le test 2 dépend du test 1 de **deux** façons : la variable, et le panier qu'il croit rempli.

Le symptôme trompeur

La suite passe parfois en entier. Elle échoue dès qu'on lance un test seul, qu'on change l'ordre, ou qu'on parallélise.

Le correctif

```
await page.request.post('/api/panier',
  { data: { id: 1 } });
```

Le test crée lui-même son état. Par API : préparer par l'interface, c'est tester deux fois la même chose.

La connexion dans un Page Object

```
export class ConnexionPage {
  constructor(private readonly page: Page) {
    this.email = page.getByLabel('Adresse email');
    this.motDePasse = page.getByLabel('Mot de passe');
    this.valider = page.getByRole('button',
      { name: 'Se connecter' });
  }

  async seConnecter(email: string, mdp: string) {
    await this.page.goto('/connexion');
    await this.email.fill(email);
    await this.motDePasse.fill(mdp);
    await this.valider.click();
  }
}
```

La ligne à ne pas franchir

Le Page Object décrit **comment interagir**, jamais ce qui doit être vrai. Les assertions restent dans les tests.

Le vrai gain

Le parcours de connexion apparaissait dans trois tests. Le jour où le formulaire gagne un champ, on corrige **un** endroit.

Dix fois vert, et le prix de l'instabilité

```
node lancer.js tp3-etapes --repeat-each=10
```

Pourquoi dix et pas une : un test instable **passé la plupart du temps**. Une exécution verte ne prouve rien, c'est la répétition qui prouve.

	RÉSULTAT	DURÉE
tp3	4 rouges + 1 intermittent	~2 min
tp3-corrige	7 verts	~6 s

Deux minutes contre six secondes

Pour une couverture équivalente. Une suite mal écrite n'est pas seulement instable : elle est **quinze fois plus lente**, parce que les attentes fixes se paient à chaque exécution.

Le livrable

Un tableau : pour chaque test, la **cause** identifiée et le **correctif** appliqué. C'est ce qu'ils présenteront à leur équipe.

TRAVAUX PRATIQUES 4

Industrialiser : mocks, fixtures, CI

La synthèse des deux jours. La dernière étape est la plus importante : une CI qui ne devient jamais rouge ne protège de rien.

Trois états qu'on ne sait pas produire autrement

```
// cas nominal : un catalogue maîtrisé
await page.route('**/api/produits*', (route) =>
  route.fulfill({ json: [{ id: 1, nom: 'Produit factice',
                        prix: 12.5, stock: 3 }] }));

// liste vide
route.fulfill({ json: [] });

// erreur 500
route.fulfill({ status: 500, json: { erreur: '...' } });
```

Pourquoi c'est visible ici

Le catalogue est rendu **par le navigateur** depuis `/api/produits`. Le mock change ce qui s'affiche à l'écran, pas seulement ce que voit l'assertion.

Ce que le mock ne doit pas devenir

Il sert à atteindre des états **difficiles à produire**. Un test qui mocke tout ne teste plus que ses propres mocks.

La fixture `pageConnectee`

```
const test = base.extend<Fixtures>({
  pageConnectee: async ({ page }, use) => {
    const r = await page.request.post('/api/connexion', {
      data: { email: 'client@demo.test', mdp: 'demo' },
    });
    expect(r.ok()).toBeTruthy();
    await page.goto('/');

    await use(page);

    // s'exécute même si le test a échoué
    await page.request.delete('/api/panier');
  },
});
```

Ce qu'une fixture apporte de plus qu'une fonction

Elle s'exécute **avant** le test, et son nettoyage **après** — y compris quand le test échoue. Un `afterEach` ne sait pas lequel a réellement créé la donnée.

Par API, pas par le formulaire. Préparer par l'interface, c'est tester la connexion une fois de plus **à chaque test**.

Mesurer, et connaître la limite

VOIE	DURÉE MESURÉE
Par le formulaire	≈ 300 ms
Par API	≈ 200 ms

L'écart paraît modeste : les deux tests chargent ensuite la page. Ce qui est économisé, c'est le **chargement du formulaire et trois interactions** — sur une authentification d'entreprise, cela se compte en secondes.

Faites la mesure devant eux plutôt que d'annoncer un chiffre.

Pourquoi ce projet n'utilise pas `storageState` partout

L'application garde le panier **en session serveur**. Partager un `storageState`, c'est partager le panier — et recréer le défaut « état partagé » du module 3, par une bonne intention.

La règle : `storageState` pour des tests en *lecture seule* ; une session par test dès qu'il écrit.

Le même fichier, trois moteurs

```
projects: [  
  { name: 'chromium', use: { ...devices['Desktop Chrome'] } },  
  { name: 'firefox', use: { ...devices['Desktop Firefox'] } },  
  { name: 'webkit', use: { ...devices['Desktop Safari'] } },  
]
```

Rien à changer dans le test : tout se passe dans la configuration.

```
node lancer.js tp4           # Chromium seul  
node lancer.js tp4-navigateurs # les trois moteurs
```

Le piège de configuration

Sans `testMatch` sur ces projets, ils ramassent **toutes** les specs — y compris celles qui dépendent d'une session préparée sur Chromium seul. On obtient des échecs qui n'ont rien à voir avec le navigateur.

Ce que ça rapporte vraiment

Sur cette application, activer Firefox a sorti **deux bugs réels** invisibles sur Chromium.

La dernière étape est la plus importante

```
webServer: {  
  command: 'node ../app/serveur.js',  
  url: 'http://localhost:3000/api/sante',  
  reuseExistingServer: !process.env.CI,  
}
```

C'est cette clé qui rend le pipeline si court : rien à orchestrer, Playwright démarre et arrête l'application lui-même.

Les workflows prêts à copier sont dans `ci/`. Sans dépôt ni runner :

```
node lancer.js ci-locale
```

Une CI qui ne devient jamais rouge ne protège de rien

Cassez volontairement l'application, poussez, et **vérifiez le rouge**. Tant que ce n'est pas fait, vous ne savez pas si le pipeline exécute vraiment vos tests.

Publiez le rapport en **artefact** et le JUnit XML : le premier sert à celui qui corrige, le second à ceux qui pilotent.

Ce que chacun a installé

TP LE RÉFLEXE À GARDER

- 1 L'assertion attend. La lecture faite à la main photographie.
- 2 Vert ne veut pas dire bon. L'IA écrit vite, la relecture n'est pas optionnelle.
- 3 On traite la cause, jamais le symptôme. Et une exécution verte ne prouve rien.
- 4 Une CI qui ne devient jamais rouge ne protège de rien.

Rejouer la progression

```
node lancer.js tp          les quatre  
node lancer.js tp2 --ui    un seul
```

Chaque fichier de `demo/tests/tp/` est un test à un moment de sa construction.

Les `x` dans la sortie

Certains tests sont annotés `test.fail()` : ils **doivent** échouer et comptent comme réussis. Si l'un se mettait à passer, Playwright le signalerait.